SQL Data Warehouse Project: Full Walkthrough and Interview Guide

Prepared by Akhil. Part A explains every script and what it fixes, with numbers I verified by running the logic on your CSVs. Part B covers gold layer, bugs, and interview prep.

Part A: Project flow and every script

1. What the project is, in plain words

A retail/bike company has two operational systems: a **CRM** (customers, products, sales orders) and an **ERP** (extra customer attributes, customer location, product categories). Both export CSV files. Analysts cannot answer "sales by country, by product category" because the data is split across six files with inconsistent formats and dirty values.

You built a **data warehouse on SQL Server** using the **Medallion architecture**:

Layer	Contents	Object type	Load	Transformations
Bronze	Raw copy of the CSVs, as-is	Tables	Batch, full load, truncate and insert	None
Silver	Cleansed, standardised data	Tables	Batch, full load, truncate and insert	Cleansing, standardisation, normalisation, derived columns, enrichment
Gold	Business-ready star schema	**Views**	No load (views read silver)	Integration, aggregation, business logic

Consumers: BI tools (Power BI, Tableau), ad-hoc SQL, machine learning.

Requirements (your screenshots 1 and 2): SQL Server; import from ERP and CRM CSVs; cleanse data quality issues before analysis; integrate both sources into one user-friendly model for analytical queries; focus on the **latest dataset only, no historisation**; document the model for business and analytics teams.

Why each requirement matters later: "latest only" is why truncate-and-reload is acceptable. "Cleanse" is the whole silver layer. "Integrate" is the gold star schema. "Document" is the naming doc, data catalog and diagrams.

2. End-to-end flow

1. Analyse requirements, design the architecture, choose layers (screenshots 1 to 3).
2. Agree naming conventions, create the database and three schemas.
3. **Bronze:** interview the source owners, create tables, bulk-load CSVs with a stored procedure.

4. **Silver:** profile the bronze data, find anomalies, create tables, write cleansing logic in a stored procedure, validate with quality checks.
5. **Gold:** study how tables relate, build `dim_customers`, `dim_products`, `fact_sales` as views, validate keys and referential integrity, write the data catalog.

3. Verified results at a glance

Table	Bronze rows	Silver rows	Why it changed
crm_cust_info	18,494	18,484	4 null-ID rows removed, 6 older duplicate versions removed
crm_prd_info	397	397	Rows kept (history), but dates and codes repaired
crm_sales_details	60,398	60,398	Rows kept, values repaired
erp_cust_az12	18,484	18,484	Values repaired
erp_loc_a101	18,484	18,484	Values repaired
erp_px_cat_g1v2	37	37	Already clean, copied

Gold: `dim_customers` 18,484 rows; `dim_products` 295 rows (current version of each product only); `fact_sales` 60,398 rows; 0 orphan fact rows; 0 duplicate surrogate keys.

---

4. `naming_conventions.md`

Purpose: one shared rulebook so every object is predictable. Without it, six months later nobody knows whether it is `CustInfo`, `cust_info` or `customer_information`.

Rules in your file:

- **snake_case, lowercase, English, no reserved words.**
- **Bronze and silver:** `<sourcesystem>_<entity>`, entity name identical to the source table (`crm_cust_info`). Keeping the source name makes lineage trivial: you can find any silver table's origin by name.
- **Gold:** `<category>_<entity>` with business-friendly names (`dim_customers`, `fact_sales`, `report_*`). Gold is for business users, so it is renamed.
- **Surrogate keys:** `<table>_key` (`customer_key`).
- **Technical columns:** `dwh_` prefix (`dwh_create_date`), meaning system-generated metadata, not business data.
- **Stored procedures:** `load_<layer>`.

Cross-question: *Why different naming in gold?* Bronze/silver are engineering layers where traceability matters; gold is the consumption layer where clarity for analysts matters.

5. `init_database.sql`

```
USE master;
CREATE DATABASE DataWarehouse;
USE DataWarehouse;
CREATE SCHEMA bronze; CREATE SCHEMA silver; CREATE SCHEMA gold;
```

- `USE master` is required because you cannot drop or create a database while connected to it.
- `CREATE DATABASE DataWarehouse` creates the container.
- Three **schemas** are logical folders inside one database. They separate layers, let you grant permissions per layer (analysts read only `gold`), and keep names like `crm_cust_info` unique across layers (`bronze.crm_cust_info` vs `silver.crm_cust_info`).
- `GO` is a batch separator used by SSMS/sqlcmd. `CREATE SCHEMA` must be the first statement in its batch, which is why each has its own `GO`.

Known flaw: the header says it drops an existing database, but the code has no drop logic, so a second run fails with "database already exists". Fix:

```
IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'DataWarehouse')
BEGIN
    ALTER DATABASE DataWarehouse SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE DataWarehouse;
END;
```

6. Source system interview (screenshot 4)

Before touching data you ask the source owners questions. Memorise the four groups:

1. **Business context and ownership:** who owns the data, what business process it supports, system and data documentation, data model and catalog. *Why:* you need someone accountable for questions like "does a negative price mean a refund?"
2. **Architecture and technology stack:** how data is stored (SQL Server, Oracle, AWS, Azure), what integration options exist (API, Kafka, file extract, direct DB). *Why:* decides how you extract.
3. **Extract and load:** incremental vs full load, scope and historical needs, expected extract size, volume limits, how to avoid hurting source performance, authentication and authorisation (tokens, SSH keys, VPN, IP whitelisting). *Why:* decides your load pattern. Here the answers were: CSV files, full load, latest data only.

7. `2_ddl_bronze.sql`

Pattern repeated for six tables:

```
IF OBJECT_ID('bronze.crm_cust_info','U') IS NOT NULL DROP TABLE bronze.crm_cust_info;
GO
CREATE TABLE bronze.crm_cust_info ( cst_id INT, cst_key NVARCHAR(50), ... );
```

- `OBJECT_ID(name,'U')` returns the object id if a user table exists. The `IF ... DROP` makes the script **re-runnable (idempotent)**.
- Columns mirror the CSV headers exactly. Bronze is the "raw, untouched" layer: **no transformations, no renaming, no cleaning**. That gives you a faithful copy to reprocess from if silver logic changes.

- Types are chosen from profiling the files. Note `sls_order_dt INT` in `crm_sales_details`: the source stores dates as numbers like `20101229`, so bronze keeps them as INT and silver converts. Note `prd_start_dt DATETIME` in bronze vs `DATE` in silver.
- No primary keys or constraints in bronze on purpose: a constraint would reject dirty rows (duplicate IDs, nulls) and you would lose raw data.

The six tables and their meaning

Table	System	Content
crm_cust_info	CRM	Customer master (id, key, name, marital status, gender, create date)
crm_prd_info	CRM	Current and historical product info (cost, line, start/end dates)
crm_sales_details	CRM	Transactional sales orders
erp_cust_az12	ERP	Extra customer info: birthdate, gender
erp_loc_a101	ERP	Customer country
erp_px_cat_g1v2	ERP	Product categories, subcategories, maintenance flag

8. `3_proc_load_bronze.sql`

```
CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN
  DECLARE @start_time DATETIME, @end_time DATETIME, @batch_start_time DATETIME,
@batch_end_time DATETIME;
  BEGIN TRY
     TRUNCATE TABLE bronze.crm_cust_info;
     BULK INSERT bronze.crm_cust_info FROM '...\\cust_info.csv'
       WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
     ...
  END TRY
  BEGIN CATCH ... END CATCH
END
```

Why a stored procedure: loading six tables by hand every day is repetitive and error-prone. A procedure packages the whole load into one repeatable call: `EXEC bronze.load_bronze;`.

Line by line:

- `CREATE OR ALTER` creates the procedure or updates it if it exists (no need to drop first).
- **Variables** `@start_time`, `@end_time` measure each table; `@batch_*` measure the whole run. `DATEDIFF(second, start, end)` computes the duration, printed for monitoring and performance tracking.
- `TRUNCATE TABLE` empties the table fast (minimally logged, resets identity, cannot be used with a `WHERE`). This implements **full load, truncate and insert**: every run replaces everything, so you never get duplicates from re-running.

- `BULK INSERT` is SQL Server's fastest way to load a flat file. `FIRSTROW = 2` skips the header row. `FIELDTERMINATOR = ','` is the CSV delimiter. `TABLOCK` takes a table-level lock which allows minimal logging and faster loads.
- Empty CSV fields land as NULL in bronze (this is why `prd_cost` and `cst_gndr` have NULLs).
- `PRINT` statements are lightweight logging so you can see progress.
- **`TRY...CATCH`**: if any statement fails, control jumps to CATCH, which prints `ERROR_MESSAGE()`, `ERROR_NUMBER()`, `ERROR_STATE()`. Without it the procedure dies with an unreadable error mid-way.

Problems to fix (and to mention in interviews as improvements):

1. CATCH prints "Error Message" label three times (should be Message, Number, State) and **swallows** the error. A scheduler would think the load succeeded. Add `THROW;` at the end of CATCH.
2. `EXEC bronze.load_bronze;` sits at the bottom of the file. Re-running the file always reloads everything. Keep definition and execution separate.
3. Absolute `D:\\...` paths are hard-coded. Better: a parameter or config table.
4. Truncate-then-load means if the load fails midway, bronze is left empty or partial. Production fix: load into a staging table, then swap, or wrap in a transaction.

Cross-question: *Why truncate instead of delete?* Faster, less logging, resets state. *Why not incremental?* Requirement says latest data only, and datasets are small. With larger data you would use watermark or CDC.

9. `4_ddl_silver.sql`

Same drop-and-create pattern, but with improvements over bronze:

- `crm_prd_info` gains **`cat_id`** (derived from the product key, used to join to ERP categories) and its dates change to `DATE`.
- `crm_sales_details` date columns change from INT to **DATE**.
- Every table gets **`dwh_create_date DATETIME2 DEFAULT GETDATE()`**.

Why `dwh_create_date` (your explanation is right): (1) **auditing and lineage**, when did this row enter the warehouse, how fresh is it; (2) **troubleshooting**, isolate rows loaded in a specific run; (3) a **reference point for future incremental logic**.

Two honest caveats: the column is filled automatically by the `DEFAULT`, so the insert statements never mention it. And because silver is truncated and fully reloaded, every row in a run gets practically the same timestamp, so today it is lineage metadata, not a watermark. In an incremental design you would keep it and filter `WHERE dwh_create_date > last_run`.

Cross-question: *Why DATETIME2 instead of DATETIME?* Higher precision and wider range, recommended by Microsoft for new work.

Header typo: the purpose line says "bronze Tables"; it should say silver.

10. Data exploration and integration (screenshot 5)

You explored how tables relate before designing silver and gold:

- `crm_sales_details.sls_prd_key` links to `crm_prd_info.prd_key`.
- `crm_sales_details.sls_cust_id` links to `crm_cust_info.cst_id`.
- `crm_cust_info.cst_key` links to `erp_cust_az12.cid` and `erp_loc_a101.cid`.
- `crm_prd_info.prd_key` links to `erp_px_cat_g1v2.id`, but only after extracting the category part of the key.

None of these join cleanly in raw form, which explains several silver transformations:

- ERP `cid` has a `NAS` prefix (11,042 rows) or dashes (all 18,484 location rows), so they do not equal `cst_key` (e.g. `AW00011000`).
- CRM product key is `CO-RF-FR-R92B-58`: first 5 chars `CO-RF` are the category (ERP uses `CO_RF`), the rest `FR-R92B-58` is the real product key that sales uses.

So silver is where you make keys joinable.

11. `quality_checks_silver.sql`, query by query

Idea: a quality check is a query that should return **no rows** if the data is good. You run it on bronze to discover problems, design the fix, then run it on silver to prove the fix worked.

Check	What it detects	Why it matters	Verified on bronze
`GROUP BY cst_id HAVING COUNT(*) > 1 OR cst_id IS NULL`	Duplicate or NULL primary keys	A PK must be unique and non-null, otherwise joins multiply rows	4 NULL ids; 6 extra duplicate rows (e.g. id 29466 appears 3 times)
`cst_key != TRIM(cst_key)`, also on names	Hidden leading/trailing spaces	`'Jon '` fails equality and group-by	17 first names and 22 last names with spaces
`SELECT DISTINCT cst_marital_status`	Cardinality and coding of low-cardinality fields	Shows `M/S/NULL`, to be mapped to Married/Single/n/a	M, S, NULL
prd_id duplicate check	Same as above for products		none
`prd_cost < 0 OR NULL`	Missing or negative cost	Cost feeds margin analysis	2 NULL costs
`DISTINCT prd_line`	Codes	`R, M, S, T, NULL` need readable names	17 NULLs; values also had trailing spaces
`prd_end_dt <`	Illegal date order	End before start is	**200 rows**

Check	What it detects	Why it matters	Verified on bronze
`prd_start_dt`		impossible	
due-date validity (`<= 0`, `LEN != 8`, range 1900 to 2050)	Dates stored as INT that are not real dates	`0` or short numbers cannot be cast	none for due date; 19 bad order dates
`sls_order_dt > sls_ship_dt OR > sls_due_dt`	Order after shipping or due	Impossible timeline	0 rows
`sales != quantity * price`, NULL or `<= 0`	Business-rule consistency	Revenue must equal qty times price	35 rows
`bdate < '1924-01-01' OR > GETDATE()`	Impossible birthdates	Age analysis breaks	15 older than 1924, 16 in the future
`DISTINCT gen`	Gender coding	Raw: `Male, Female, F, M, NULL, blanks`	many variants
`DISTINCT cntry`	Country coding	Raw: `DE, US, USA, United States, Germany, blank, NULL`	yes
`TRIM` checks on `erp_px_cat_g1v2`	Spaces		none, already clean

Note: the due-date check in your file queries `bronze.crm_sales_details`, not silver. It is a diagnostic on raw data because after conversion to DATE the INT-based tests are meaningless. Mention that deliberately if asked.

12. `5_proc_load_silver.sql`, transformation by transformation

Same skeleton as the bronze procedure (variables, timing, TRY/CATCH, truncate then insert), but each insert is `INSERT ... SELECT` from bronze with transformations.

12.1 `crm_cust_info`

```
SELECT ... FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
  FROM bronze.crm_cust_info WHERE cst_id IS NOT NULL
) t WHERE flag_last = 1;
```

- **Problem:** duplicate customer ids because the CRM keeps several versions of a customer. Verified: id 29466 has three rows (create dates Jan 25, 26, 27; the early ones lack names).
- **Solution:** `ROW_NUMBER` numbers rows within each customer, newest first; keep number 1. `WHERE cst_id IS NOT NULL` drops the 4 rows with no key, because a row with no identity cannot be joined to sales.

- **Why ROW_NUMBER and not DISTINCT or GROUP BY:** rows differ in content, so DISTINCT does not collapse them; ROW_NUMBER lets you choose which duplicate wins.
- `TRIM(cst_firstname)`: removes spaces.
- `CASE` maps `S` to Single, `M` to Married, else `n/a`; same for gender `F/M`. `UPPER(TRIM())` makes the mapping robust to lowercase and spaces. **Normalisation** to readable values; the `ELSE 'n/a'` is a deliberate choice to avoid NULLs in reporting, documented as "unknown".
- Result: 18,484 rows, one per customer.

12.2 `crm_prd_info`

- `REPLACE(SUBSTRING(prd_key,1,5),'-','_') AS cat_id`: extracts category id and aligns format with ERP (`CO-RF` becomes `CO_RF`). **Derived column** that enables the join to `erp_px_cat_g1v2`.
- `SUBSTRING(prd_key,7,LEN(prd_key)) AS prd_key`: strips the 6-char prefix so it matches `sls_prd_key` in sales.
- `ISNULL(prd_cost,0)`: replaces missing cost with 0 (a business decision; alternative is leaving NULL).
- `CASE` on `prd_line`: `M` to Mountain, `R` to Road, `S` to Other Sales, `T` to Touring, else `n/a`. 17 rows become `n/a`.
- `CAST(prd_start_dt AS DATE)`: drops the time part.
- **`prd_end_dt` rebuilt with `LEAD`:** the source end dates were wrong (200 rows had end before start). Instead of trusting them, end date = next version's start date minus one day, per product. Verified example for helmet `HL-U509`: starts 2011-07-01, 2012-07-01, 2013-07-01 became ends 2012-06-30, 2013-06-30, NULL. **NULL end date means "current version"**, and gold relies on that.
- The window is `PARTITION BY prd_key ORDER BY prd_start_dt`. In the SELECT list you alias a transformed `prd_key`, but window clauses are evaluated on the source column (the full key), which is what you want. Be able to say this.
- 397 rows kept (this is a history table), 295 distinct products.

12.3 `crm_sales_details`

- **Dates:** `CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE) END`. INT `20101229` cannot be cast straight to DATE, so INT to VARCHAR to DATE; invalid values (0 or wrong length) become NULL. Verified: 19 order dates are `0`, none for ship or due. Those 19 stay NULL in silver and gold (I confirmed 19 fact rows with NULL `order_date`).
- **Sales:** if `sls_sales` is NULL, `<= 0`, or not equal to `quantity * ABS(price)`, recompute as `quantity * ABS(price)`. **Price:** if NULL or `<= 0`, derive as `sales / NULLIF(quantity,0)`. `NULLIF(x,0)` avoids divide-by-zero.
- **Business rule encoded:** `Sales = Quantity x Price` (from the data model image). Verified problem rows: 8 NULL sales, 5 non-positive sales, 7 NULL prices, 5 negative prices (e.g. price `-769`), 35 rows total. After the fix: 0 rows fail the consistency check.
- **Caveat to say proactively:** using `ABS(price)` assumes negatives are data-entry sign errors. If negatives meant refunds, this would corrupt revenue. In a real project you confirm with the data owner (back to the source system interview).

12.4 `erp_cust_az12`

- `CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid,4,LEN(cid)) ELSE cid END`: removes the `NAS` prefix so `NASAW00011000` becomes `AW00011000`, matching CRM `cst_key`. Verified: 11,042 rows had the prefix.
- `bdate > GETDATE()` set to NULL: 16 future birthdates are impossible. (The check file also flags dates before 1924; the procedure does not null those 15, so they pass through. Know this gap.)
- Gender: `UPPER(TRIM(gen)) IN ('F','FEMALE')` to Female, `('M','MALE')` to Male, else `n/a`. Raw data had `Male, Female, F, M, NULL, empty, whitespace` variants. Result: Male 8,612, Female 8,396, n/a 1,476.

12.5 `erp_loc_a101`

- `REPLACE(cid,'-','')`: `AW-00011000` becomes `AW00011000`.
- Country: `DE` to Germany; `US` and `USA` to United States; blank or NULL to `n/a`; everything else trimmed. Verified: raw had `US` 1,500 and `USA` 2,591 beside `United States` 3,391; after cleaning United States is 7,482 and Germany 1,780 (1,214 + 566). `n/a` is 337 (332 NULL plus 5 blank/space).

12.6 `erp_px_cat_g1v2`

No transformation needed; straight copy. The quality checks proved it was clean (no spaces, maintenance is only Yes/No). It is still loaded to silver for **consistency**: downstream layers only read silver.

12.7 Techniques used in silver, by name

Data cleansing (remove duplicates, nulls, trim), **standardisation/normalisation** (code to readable value), **handling missing data** (`ISNULL`, `n/a`, NULL), **derived columns** (`cat_id`, recalculated end date, recalculated sales/price), **type casting** (INT to DATE, DATETIME to DATE), **data enrichment** (adding category key), **business-rule validation** (sales = qty x price), **window functions** (`ROW_NUMBER`, `LEAD`).

Header typo: the silver procedure's CATCH prints `ERROR OCCURED DURING LOADING BRONZE LAYER`. Copy-paste slip; change to SILVER, and add `THROW;`.

Part B: Gold layer, bugs, and interview preparation

13. Gold layer: the logic (reconstructed, see note at top)

Purpose: silver is clean but still shaped like the source systems (six tables, odd keys). Gold reshapes it into a **star schema** that analysts can query with two or three simple joins. Gold objects are **views**: no storage, no load procedure, always reflect the current silver data.

13.1 Dimensions vs facts

- **Fact table:** records events/measurements (numbers you sum or count): `fact_sales` holds quantity, price, sales amount. Many rows, mostly keys and numbers.

- **Dimension table:** describes the context of the event (who, what, where): `dim_customers`, `dim_products`. Fewer rows, many descriptive text columns.
- **Star schema:** one central fact joined to dimensions, like a star. Few joins, simple, fast for BI.
- **Snowflake schema:** dimensions further normalised into sub-tables (e.g. product to subcategory to category tables). Saves storage, but more joins and harder for business users. You chose star because simplicity and query speed matter more than storage here.

13.2 `gold.dim_customers`

```
CREATE VIEW gold.dim_customers AS
SELECT
  ROW_NUMBER() OVER (ORDER BY ci.cst_id) AS customer_key,
  ci.cst_id AS customer_id, ci.cst_key AS customer_number,
  ci.cst_firstname AS first_name, ci.cst_lastname AS last_name,
  la.cntry AS country, ci.cst_marital_status AS marital_status,
  CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr
       ELSE COALESCE(ca.gen,'n/a') END AS gender,
  ca.bdate AS birthdate, ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;
```

- **Data integration:** three silver tables (one CRM, two ERP) become one customer entity. CRM `crm_cust_info` is the **master** and the other two are joined to enrich it.
- **LEFT JOIN** keeps every CRM customer even if ERP has no matching row. An INNER JOIN would silently drop customers. (Verified: every CRM customer matched in both ERP tables, 0 missing.)
- **Gender rule:** CRM is treated as the master source; ERP is the fallback when CRM says `n/a`. Verified result: Male 9,341, Female 9,128, only 15 `n/a`. Before the fallback, CRM alone had 4,578 NULL genders, so the fallback recovers about 4,500 customers.
- **Data conflict you should know about:** for 57 customers CRM and ERP both have a gender and they disagree (40 CRM Female / ERP Male, 17 the reverse). The rule "CRM wins" is a business decision that must be documented and agreed with the data owner.
- **Surrogate key:** `ROW_NUMBER()` generates `customer_key`, a warehouse-owned integer key independent of source ids. Reasons: source keys may change or collide across systems; integer keys join faster; it decouples the model from source systems.
- **Renaming:** `cst_firstname` to `first_name` etc. Business-friendly names, following the gold naming rule.
- **Quality check after joining:** joins can create **duplicates** (one-to-many). Verified: 18,484 rows after joins equals the 18,484 customers, so no fan-out.

13.3 `gold.dim_products`

```
CREATE VIEW gold.dim_products AS
SELECT ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,
  pn.prd_id AS product_id, pn.prd_key AS product_number, pn.prd_nm AS product_name,
  pn.cat_id AS category_id, pc.cat AS category, pc.subcat AS subcategory,
  pc.maintenance AS maintenance, pn.prd_cost AS cost, pn.prd_line AS product_line,
  pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE pn.prd_end_dt IS NULL;   -- current products only
```

- **`WHERE prd_end_dt IS NULL`** keeps only the current version of each product. This is why 397 silver rows become 295 gold rows (one per product). It is the implementation of the requirement "latest data only, no historisation". The `LEAD` logic in silver is what makes this filter possible.
- Join to ERP categories using the derived `cat_id`; this is where the silver key engineering pays off.
- **Data issue found:** 7 pedal products have `cat_id = CO_PE`, which does not exist in the ERP category table, so their category, subcategory and maintenance are NULL in `dim_products` (`ML Mountain Pedal`, `Touring Pedal`, `LL Road Pedal`, etc.). A LEFT JOIN keeps them visible instead of dropping them. In real life you raise this with the ERP owner.
- Maintenance only has `Yes`/`No` (from the model image).

13.4 `gold.fact_sales`

```
CREATE VIEW gold.fact_sales AS
SELECT sd.sls_ord_num AS order_number, pr.product_key, cu.customer_key,
  sd.sls_order_dt AS order_date, sd.sls_ship_dt AS shipping_date, sd.sls_due_dt AS
due_date,
  sd.sls_sales AS sales_amount, sd.sls_quantity AS quantity, sd.sls_price AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products  pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;
```

- This is the **dimension lookup**: replace natural/business keys (`sls_prd_key`, `sls_cust_id`) with the dimensions' **surrogate keys**. That is how fact connects to dimensions in a star schema.
- Verified: 60,398 rows, same as silver; 0 rows with a missing product or customer key.
- `sales_amount = quantity x price` was enforced in silver. 19 rows have NULL `order_date` (source value `0`).

13.5 Verified sample business answers

Sales by category: Bikes about 28.3M, Accessories about 0.70M, Clothing about 0.34M. Sales by country: United States about 9.16M, Australia about 9.06M, United Kingdom about 3.39M, Germany 2.89M, France 2.64M, Canada 1.98M, `n/a` about 0.23M. Be ready to state that these needed all three layers working: country comes from the ERP location table, category from the ERP category table, sales from CRM.

13.6 `quality_checks_gold.sql`

1. **Uniqueness of `customer_key` and `product_key`** (`GROUP BY key HAVING COUNT(*) > 1`). A surrogate key must be unique. Result: empty.
2. **Referential integrity / model connectivity:** left join fact to both dimensions and look for rows where a dimension key is NULL. Any row returned is a sale that cannot be attributed to a customer or product. Result: 0 rows.
3. Note: the header comment says `gold.product_key`; the object is `gold.dim_products`. Cosmetic slip.

13.7 `data_catalog.md`

A data catalog documents each gold table: purpose, column names, data types and business description. Audience: business users and analysts who never read your SQL. It fulfils the "clear documentation" requirement and makes the warehouse self-service. Two cells in your catalog are cut off (`create_date` and `start_date` descriptions end mid-sentence); finish them before showing it to anyone.

Small issues in the model image: label `material_status` should be `marital_status`, and `country` appears twice in `dim_customers`.

---

14. Master list of issues in your scripts (fix before the interview)

#	File	Issue	Fix
1	init_database	No drop logic despite header	`IF EXISTS` + `SINGLE_USER` + `DROP`
2	load_bronze / load_silver	CATCH swallows errors	add `THROW;` at the end of CATCH
3	both CATCH blocks	Labels all say "Error Message"	Message / Number / State
4	load_silver	CATCH says BRONZE	change to SILVER
5	3_proc_load_bronze	`EXEC` at the bottom of the definition file	move to a separate run script
6	4_ddl_silver	Header says bronze	change to silver
7	quality_checks_silver	Due-date check reads bronze	label it as a bronze diagnostic
8	load_silver	`ABS(sls_price)` is a business assumption	confirm negative price meaning
9	load_silver	Birthdates before 1924 are checked but not nulled	decide and align check and procedure
10	gold	`ROW_NUMBER` surrogate keys are not stable across reloads	see interview Q below
11	data_catalog	Truncated descriptions	complete them

Part C: How to explain the project in an interview

15. Be straight about origin

This project follows the Data With Baraa tutorial. Interviewers know these tutorials. If you claim you invented the requirements or the architecture, one follow-up question exposes it, and you lose credibility on everything else. Say you **built it end to end following a guided project, and can explain and defend every decision**, then add your own contribution (the fixes in section 14, an incremental-load design, a stable surrogate key). That is stronger than pretending, because every question in this document becomes an opportunity to show you understand rather than copied.

16. The 60-second answer

"I built a data warehouse in SQL Server for a sales business. Data came from two source systems, a CRM and an ERP, as six CSV files. I used a medallion architecture. Bronze holds raw copies loaded with a BULK INSERT stored procedure with error handling and load timing. Silver cleans and standardises the data with another stored procedure: removing duplicate customers with ROW_NUMBER, fixing product end dates with LEAD, repairing invalid dates and sales values, standardising gender and country codes, and making keys joinable across CRM and ERP. Gold is a star schema of views: customer and product dimensions plus a sales fact, with surrogate keys. I validated each layer with SQL quality checks and documented the model with a data catalog and naming conventions. The result lets analysts answer questions like sales by country or category with simple joins."

17. The 3-minute structured version

1. **Business problem:** sales data split across CRM and ERP, dirty and inconsistent; need one trustworthy model for reporting.
2. **Architecture:** medallion, three layers, and why (separation of concerns, reprocessing from raw, quality gates).
3. **Bronze:** raw, no transformations, full load, truncate and insert, `BULK INSERT`, try/catch and timing.
4. **Silver:** list three or four concrete issues with numbers (4 null IDs and 6 duplicate customer rows; 200 products with end before start; 19 invalid order dates; 35 inconsistent sales rows; 11,042 prefixed ERP IDs) and how each was fixed.
5. **Gold:** star schema, surrogate keys, CRM-first gender rule, current-only products, fact lookups.
6. **Validation and documentation:** quality checks, data catalog, naming conventions.
7. **Limitations and improvements:** incremental loads, scheduling, stable keys, data-owner decisions.

Use numbers. Numbers make it sound like you ran it.

18. Techniques and concepts to name (and be able to explain)

Architecture and design: medallion architecture, layered ETL, ELT-style SQL transformations, star schema vs snowflake, fact vs dimension, surrogate vs natural keys, data lineage, naming conventions, data catalog.

Loading: full load vs incremental, truncate and insert, `BULK INSERT`, idempotent scripts, batch processing.

T-SQL features: stored procedures, `TRY...CATCH`, `TRUNCATE`, `DATEDIFF`, `GETDATE`, `OBJECT_ID`, `CREATE OR ALTER`, `DEFAULT` constraints, window functions (`ROW_NUMBER`, `LEAD`), `CASE`, `COALESCE`/`ISNULL`, `NULLIF`, `TRIM`, `SUBSTRING`, `REPLACE`, `CAST`, joins (LEFT), views, subqueries.

Data quality: duplicate and null key checks, whitespace, standardisation, range and date-order validation, business-rule validation (sales = qty x price), referential integrity checks, metadata columns.

19. Cross-questioning bank, with answers

Architecture

1. *Why medallion and not load straight to a model?* Raw data kept separately lets you re-run transformations without re-extracting; each layer has a single responsibility; problems are easier to trace; you can add consumers at the right layer.
2. *Why not clean in bronze?* Bronze must mirror the source so you can prove what arrived and reprocess if logic changes.
3. *Why are gold objects views and not tables?* Data is already clean and small; a view always reflects silver, needs no load job. Trade-off: computed on every query. With large data or slow queries you materialise to tables or indexed/aggregated tables (`report_*`).
4. *ETL or ELT?* Closer to ELT: data is loaded raw into the warehouse, then transformed in SQL inside the warehouse.
5. *Why schemas instead of separate databases?* Simpler for a project this size; schema-level permissions give separation. Large organisations might use separate databases.
6. *Why no primary keys in bronze?* Constraints would reject dirty rows and lose raw data.

Loading

7. *What if the bronze load fails halfway?* Currently tables are left partially loaded because truncate happens first and there is no transaction around it. Improve: load to staging then swap, or wrap in a transaction, and `THROW` so the scheduler sees the failure.
8. *How would you make it incremental?* Use a watermark (`create_date` or `dwh_create_date`), load only newer rows, and `MERGE` or update/insert into silver; for deletes you need CDC or a full compare. Requires the source to expose a reliable change indicator, which is a source-system-interview question.
9. *How would you schedule it?* SQL Server Agent job, or an orchestrator like Airflow or Azure Data Factory, running `load_bronze` then `load_silver`, with alerting on failure.
10. *Why TABLOCK?* Allows minimal logging and faster bulk loading by locking the whole table.
11. *Why TRUNCATE vs DELETE?* Faster, minimally logged; cannot filter and needs more permissions; fine for full reload.

Silver transformations

12. *Why ROW_NUMBER and not DISTINCT?* Duplicates have different content (different create dates and sometimes missing names); you need to pick the most recent version deterministically.

13. *How do you decide which duplicate to keep?* The latest `cst_create_date`, on the assumption that the newest record is the most complete. Verified: older versions of id 29466 lacked names and gender.
14. *Why `LEAD` to recompute end dates?* Source end dates were wrong for 200 rows. LEAD derives each version's end from the next version's start, giving contiguous, non-overlapping history. The latest version stays NULL, meaning current.
15. *Why `n/a` and not NULL?* Reporting tools show NULL as blank and group them inconsistently; `n/a` is explicit. Trade-off: it is a string in a column that could be typed, and you lose the ability to distinguish "unknown" from "not applicable".
16. *Why replace missing cost with 0?* Avoids NULL arithmetic. Risk: 0 looks like a real cost. Alternative is leaving NULL and handling it in gold.
17. *How do you know your cleansing is right?* Run the quality checks on silver and expect empty results; compare row counts; spot-check specific IDs before and after.
18. *What if sales and price are both wrong?* Current logic would not recover; you would flag the row, quarantine it into an error table, and report it to the data owner.
19. *Why did you not fix birthdates before 1924?* The procedure nulls only future dates; the check flags old ones. It is an inconsistency I would align, probably by nulling them too.
20. *Why cast INT to VARCHAR to DATE?* SQL Server cannot convert an INT like 20101229 directly to DATE; going via a string in `yyyymmdd` format works.

Gold and modelling

21. *What is a surrogate key and why use one?* A system-generated key owned by the warehouse. Natural keys can change, repeat across systems, or be non-integer; surrogate keys give stable, compact join keys.
22. *Are `ROW_NUMBER()` surrogate keys safe?* Not really: if rows are added or ordering changes, keys shift, and anything that stored an old key (reports, downstream tables) breaks. Better: an `IDENTITY` column in a physical dimension table, or a hash key of the business key, so existing keys never change. This is the best "improvement" answer in the whole project.
23. *Why LEFT JOIN?* To keep all driving-table rows even without a match; INNER JOIN would silently lose data. Then check for NULL keys.
24. *Why did you keep only current products?* Requirement: latest data only. If history were needed you would build a **slowly changing dimension type 2** with start/end dates and a current flag, and the fact would link to the version valid at order date.
25. *Which source wins when CRM and ERP disagree?* CRM is the master; ERP is only used when CRM is unknown. 57 conflicts exist; the rule must be agreed with the business.
26. *How do you check the model?* Surrogate key uniqueness, and a left-join from fact to dimensions looking for NULL keys; also row counts equal to silver.
27. *Why is the fact grain what it is?* One row per order line (order number plus product), as in the source sales details.
28. *Why denormalise?* Dimensional models trade redundancy for fewer joins and easier analytics.

Data quality and governance

29. *What was the nastiest data issue?* Pick sales: five types of corruption in one table (null, zero or negative values, mismatched totals, zero dates) fixed through one rule, qty x price, with the caveat that the ABS assumption must be validated.
30. *Where would you log bad rows?* Quarantine tables with a reason code rather than silently repairing; keep a count per load.
31. *How do you handle schema changes in the source?* Bronze loads by position, so a new column in the CSV breaks the load; you would add schema validation or use a flexible ingestion layer.
32. *How would you test this in a team?* Version-controlled scripts in Git, automated runs of quality checks after each load, and fail the pipeline if any check returns rows.

20. Questions to ask back, which make you sound senior

- "What does a negative price mean in your CRM?" (shows you caught the ABS risk)
- "Is the CRM or ERP the system of record for gender?"
- "Do you need history of product price changes, or only the current version?"

21. What to do next

1. Fix items 1 to 9 in section 14, commit them to Git, and say so in the interview.
2. Rebuild `dim_*` as physical tables with an `IDENTITY` key and a small incremental-load procedure. That one change turns this into a project that is partly yours.
3. Practise the 60-second answer out loud until it takes under a minute, then the 3-minute version.
4. Pick five numbers from section 3 and memorise them.

Send me your gold view scripts next time and I will check them against what I reconstructed here, plus the first batch of mock interview questions if you want to practise.

Part D: Silver layer from zero (beginner deep dive)

This part assumes you know nothing about what happened inside the silver layer. Read it top to bottom once. Every example row below is a real row from your CSV files.

D1. The big picture: what silver is and how we knew what to fix

Bronze is a photocopy of the CSV files, mistakes included. Silver is the same data after someone has checked it and corrected the mistakes. Nobody tells you what is wrong in the data. You **find out by asking the data questions with SQL**. That is the entire method, and it has four steps:

1. **Profile:** run small SELECT queries on the bronze table that would return rows only if something is wrong (duplicates, spaces, impossible dates). These are the "quality checks".
2. **Look at the bad rows:** read a few of them and understand why they are bad.
3. **Decide and fix:** write a transformation in the silver INSERT query that repairs that specific problem.
4. **Verify:** run the same check on the silver table. It must return nothing.

So every transformation in `5_proc_load_silver.sql` exists because a check on bronze returned something. If a transformation exists, there was a problem behind it. If you can state the problem, the check that found it, and the fix, you understand silver.

Why not fix it in bronze? Bronze must stay identical to the source, so you always have the original to compare against and to rebuild from.

How the silver procedure runs. For each table it does two things: `TRUNCATE TABLE silver.xxx` (empty it so re-running never duplicates rows) and `INSERT INTO silver.xxx (...) SELECT ... FROM bronze.xxx` (read bronze, clean on the way, write the result). The `SELECT` is where all cleaning lives. The `PRINT` lines, `DATEDIFF` timers and `TRY...CATCH` are logging and safety, not cleaning.

Toolbox: the functions you will see, in plain English

Function	What it does	Tiny example
`TRIM(x)`	Removes spaces at the start and end	`' Jon '` becomes `'Jon'`
`UPPER(x)`	Makes text capital letters so `m` and `M` compare equal	`'m'` becomes `'M'`
`CASE WHEN ... THEN ... ELSE ... END`	An if / else-if chain inside SQL	`WHEN 'M' THEN 'Married'`
`ISNULL(x, 0)`	If x is NULL, use 0	`NULL` becomes `0`
`NULLIF(x, 0)`	Returns NULL when x equals 0, otherwise x. Used to avoid dividing by zero	`10 / NULLIF(0,0)` gives NULL, not an error
`SUBSTRING(text, start, length)`	Cuts out part of a text, counting from position 1	`SUBSTRING('NASAW001', 4, 10)` gives `AW001`
`REPLACE(text, a, b)`	Swaps every `a` for `b`	`'AW-001'` becomes `'AW001'`
`LEN(x)`	Number of characters	`LEN('20130511')` is 8
`CAST(x AS type)`	Converts one data type to another	text to DATE
`ROW_NUMBER() OVER (PARTITION BY a ORDER BY b)`	Numbers rows 1, 2, 3 separately inside each group a, in order of b	see D3
`LEAD(x) OVER (PARTITION BY a ORDER BY b)`	Looks at the next row in the group and returns its x	see D4

Two ideas to hold on to: a **NULL** means "no value / unknown" (it is not zero and not an empty string), and a **window function** (`OVER (...)`) calculates something across related rows **without collapsing them** the way `GROUP BY` does.

D2. Notation used for the six tables

For each table: what it contains, how we found each problem, what the bad data looked like, the exact query fix explained line by line, and how we proved it worked.

D3. Table 1: `crm_cust_info` (customers from CRM)

What it holds: one row per customer: id, key (like `AW00011000`), first and last name, marital status code, gender code, creation date. 18,494 rows in bronze.

Problem 1: duplicate and empty customer IDs

How we found it: the check

```
SELECT cst_id, COUNT(*)
FROM bronze.crm_cust_info
GROUP BY cst_id
HAVING COUNT(*) > 1 OR cst_id IS NULL;
```

Read it like this: `GROUP BY cst_id` puts rows with the same id into one bucket; `COUNT(*)` counts how many rows are in each bucket; `HAVING COUNT(*) > 1` keeps only buckets with more than one row; `OR cst_id IS NULL` also keeps rows with no id. A customer id is supposed to be unique, so a **correct table returns nothing**. Yours returned results.

What it showed: 4 rows with no id at all (junk rows with keys like `SF566` and `PO25` and everything else empty), and customers appearing more than once. Customer 29466 appears **three times**:

cst_id	first name	last name	marital	gender	created
29466	(empty)	(empty)	(empty)	(empty)	2026-01-25
29466	Lance	Jimenez	M	(empty)	2026-01-26
29466	Lance	Jimenez	M	M	2026-01-27

Why it is a problem: if two rows share an id, then joining sales to customers matches each sale twice and **sales totals double**. A customer with no id cannot be linked to anything.

The fix:

```
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
  FROM bronze.crm_cust_info
  WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1
```

- `WHERE cst_id IS NOT NULL` throws away the 4 junk rows.
- `PARTITION BY cst_id` means "treat each customer separately".
- `ORDER BY cst_create_date DESC` means "newest record first".

- `ROW_NUMBER()` writes 1, 2, 3 next to each customer's rows. For customer 29466: Jan 27 gets 1, Jan 26 gets 2, Jan 25 gets 3.
- The outer `WHERE flag_last = 1` keeps only the number 1, the newest and most complete row.
- The inner query is wrapped in a bracket and given the alias `t` because you cannot filter on a window function in the same query where you create it; you must create it first, then filter in an outer query.

Why not `DISTINCT`? `DISTINCT` removes only rows that are identical in every column. These rows are different (different dates, some missing names), so DISTINCT would keep all three.

Proof: after the load, the same duplicate check on `silver.crm_cust_info` returns nothing and the table has 18,484 rows (18,494 minus 4 junk minus 6 older duplicates).

Problem 2: hidden spaces in names

How we found it: `WHERE cst_firstname != TRIM(cst_firstname)`. If trimming changes the value, there were spaces. 17 first names and 22 last names failed. Examples (brackets added so you can see the spaces): `[ Jon]`, `[Yang ]`, `[  Lauren]`, `[ Ian ]`.

Why it is a problem: to SQL, `'Jon'` and `' Jon'` are different values. Grouping by name, joining on name, or searching for "Jon" gives wrong results, and reports look unprofessional.

The fix: `TRIM(cst_firstname) AS cst_firstname` and the same for last name.

Problem 3: coded values nobody can read

How we found it: `SELECT DISTINCT cst_marital_status` and `SELECT DISTINCT cst_gndr`. `DISTINCT` lists each different value once, so it shows every spelling that exists. Result: marital status is `M`, `S` or NULL; gender is `M`, `F` or NULL (4,578 NULL).

Why it is a problem: `M` could mean Married or Male depending on the column. A dashboard showing "M: 10,013" confuses business users. NULLs can drop out of charts or be shown as blank.

The fix:

```
CASE WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
     WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
     ELSE 'n/a' END AS cst_marital_status
```

- `TRIM` and `UPPER` first, so `' m'` still matches. This is defensive: the current data may be neat, but tomorrow's file may not be.
- `ELSE 'n/a'` catches everything unexpected, including NULL, and gives it one clear label. The same pattern is applied to gender with `F` to Female and `M` to Male.
- This is called **normalisation / standardisation**: one concept, one spelling.

Proof: `SELECT DISTINCT` on silver now returns only `Single` and `Married` for marital status, and `Male`, `Female`, `n/a` for gender. No `n/a` appears for marital status because the 7 NULL marital values in bronze sat in the rows that were removed (the 4 junk rows and the 6 older duplicates), so the `ELSE 'n/a'` branch never fires on this data. It stays in the code as protection for future files.

D4. Table 2: `crm_prd_info` (products from CRM)

What it holds: product versions with cost, product line and validity dates. A product has **one row per version** (for example, its price changed in 2012). 397 rows representing 295 distinct products.

Problem 1: the product key mixes two things

A key looks like `CO-RF-FR-R92B-58`. It is really two keys glued together: **`CO-RF`** (the category) and **`FR-R92B-58`** (the actual product).

How we found it: in the integration diagram (screenshot 5) the sales table links to products by `prd_key` and products link to the ERP category table by `id`. We compared the values. Sales has `BK-R93R-62` style keys (no category prefix) and the ERP category table has ids like `CO_RF` (underscore, 5 characters). Neither matches `CO-RF-FR-R92B-58` directly, so **the joins would return nothing**.

The fix:

```
REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,
SUBSTRING(prd_key, 7, LEN(prd_key))         AS prd_key,
```

- `SUBSTRING(prd_key, 1, 5)` takes 5 characters starting at position 1: `CO-RF`.
- `REPLACE(..., '-', '_')` changes it to `CO_RF`, the format the ERP table uses. Result: a new column `cat_id` (a **derived column**; this is why silver DDL added `cat_id`).
- `SUBSTRING(prd_key, 7, LEN(prd_key))` starts at position 7 (position 6 is the dash) and goes to the end: `FR-R92B-58`. That now matches the sales table.

Problem 2: missing cost and product line

How we found it: `WHERE prd_cost IS NULL` returned 2 rows (the two `HL Road Frame` rows) and `SELECT DISTINCT prd_line` showed `R`, `M`, `S`, `T` plus 17 NULLs, with trailing spaces such as `'R '`.

The fix: `ISNULL(prd_cost, 0)` turns NULL cost into 0, so arithmetic like `cost * quantity` does not turn into NULL. And:

```
CASE WHEN UPPER(TRIM(prd_line)) = 'M' THEN 'Mountain'
     WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
     WHEN UPPER(TRIM(prd_line)) = 'S' THEN 'Other Sales'
     WHEN UPPER(TRIM(prd_line)) = 'T' THEN 'Touring'
     ELSE 'n/a' END AS prd_line
```

`TRIM` is what makes `'R '` match `'R'`. Without it, every Road product would fall into `ELSE`. The 17 NULLs become `n/a`.

Caveat: a cost of 0 is a decision, not a fact. 0 could be mistaken for a real free product. Be ready to say that.

Problem 3: end dates are impossible (the most important one in this table)

How we found it: `WHERE prd_end_dt < prd_start_dt`. An end before a start cannot happen. **200 of 397 rows** failed. Real example, the Red Sport-100 Helmet (`AC-HE-HL-U509-R`):

prd_id	start	end in source

prd_id	start	end in source
212	2011-07-01	**2007-12-28** (before start!)
213	2012-07-01	**2008-12-27** (before start!)
214	2013-07-01	(empty)

The three rows are three versions of the same product (cost 12, then 14, then 13). The source end dates are simply wrong.

The fix: stop trusting the source end date and compute it. Logic: a version ends the day before the next version starts.

```
CAST(
  LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) - 1
  AS DATE) AS prd_end_dt
```

- `PARTITION BY prd_key` handle each product on its own.
- `ORDER BY prd_start_dt` put its versions in time order.
- `LEAD(prd_start_dt)` is "the start date of the next row in this group". For the first helmet row that is 2012-07-01.
- `- 1` subtracts one day (in SQL Server, date minus 1 means one day earlier): 2012-06-30.
- The last version has no next row, so `LEAD` returns NULL. That is **intentional**: NULL end date means "this version is current". Gold uses this (`WHERE prd_end_dt IS NULL`) to pick the current products.
- `CAST(... AS DATE)` removes the time part. `CAST(prd_start_dt AS DATE)` does the same to the start date.

Result for the helmet: 2011-07-01 to 2012-06-30, 2012-07-01 to 2013-06-30, 2013-07-01 to NULL. Continuous, no gaps, no overlaps.

Subtle point: `LEAD ... PARTITION BY prd_key` uses the **original** full key from bronze (`AC-HE-HL-U509-R`), because the alias created in the same SELECT is not visible to the window clause. That is fine, because the full key is still unique per product.

Proof: `WHERE prd_end_dt < prd_start_dt` on silver returns 0 rows.

D5. Table 3: `crm_sales_details` (the sales orders, the most important table)

What it holds: one row per order line: order number, product key, customer id, three dates, sales amount, quantity, price. 60,398 rows. This is the data the business actually wants to analyse, so errors here directly mean wrong money.

Problem 1: dates stored as numbers, some invalid

How we found it: bronze DDL declares the date columns as `INT` because that is how they arrive: the date 11 May 2013 is stored as the number `20130511`. The check:

```
WHERE sls_due_dt <= 0 OR LEN(sls_due_dt) != 8
   OR sls_due_dt > 20500101 OR sls_due_dt < 19000101
```
asks: is any date zero, not 8 digits, or outside 1900 to 2050? Running the same test on the order date found **19 rows where the order date is `0`** (for example order `SO64338`: order date 0, ship date 20130823, due date 20130828). Ship and due dates were fine.

Why it is a problem: a number is not a date. You cannot sort by month, calculate days between order and shipping, or filter by year. And `0` is not a date at all.

The fix:

```
CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
     ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE) END AS sls_order_dt
```

- If the value is 0 or not 8 digits it cannot be a valid date, so we store **NULL (unknown)** instead of inventing a date.
- Otherwise convert in two steps: number `20130511` to text `'20130511'`, then text to a real DATE. SQL Server cannot convert the integer straight to a date, so the text step is needed.
- The same pattern is repeated for ship and due dates.

Why NULL and not a guess? Guessing a date creates false information. NULL honestly says "unknown". Those 19 rows keep their sales amounts but have no order date.

Also checked: `sls_order_dt > sls_ship_dt OR sls_order_dt > sls_due_dt`. An order cannot be placed after it ships. This returned 0 rows, so no fix needed, but the check stays as a safety net.

Problem 2: sales, quantity and price do not agree

The business rule (from the data model image): **Sales = Quantity x Price**. Any row breaking it is wrong.

How we found it:

```
WHERE sls_sales != sls_quantity * sls_price
   OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL
   OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0
```
That found **35 bad rows**, of several kinds:

Kind	Example
Price is NULL	`SO51259`: sales 10, quantity 2, price empty
Sales is NULL or zero or negative	8 NULL sales and 5 non-positive sales in the data
Price is negative	`SO57804`: sales 769, quantity 1, price **-769**
Total does not match	`SO58335`: sales 35, quantity 2, price 35. It should be 70

The fix:

```
CASE WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity *
ABS(sls_price)
       THEN sls_quantity * ABS(sls_price)
     ELSE sls_sales END AS sls_sales,

CASE WHEN sls_price IS NULL OR sls_price <= 0
```

```
      THEN sls_sales / NULLIF(sls_quantity, 0)
    ELSE sls_price END AS sls_price
```

- **Sales:** if sales is missing, not positive, or does not equal quantity x price, recalculate it as quantity x |price|. Otherwise keep the original.
- `ABS(sls_price)` is the absolute value: -769 becomes 769. This **assumes a negative price is a typing sign mistake**. If the business uses negative prices for refunds, this fix is wrong. Always say that out loud; it is a good senior-sounding point.
- **Price:** if price is missing or not positive, derive it as sales divided by quantity (`NULLIF(quantity,0)` prevents a divide-by-zero error by turning a 0 quantity into NULL).
- Order of reasoning: sales is repaired using price, and price is repaired using sales. Each rescues the other when only one is broken.

Proof: the consistency check on silver returns 0 rows.

D6. Table 4: `erp_cust_az12` (extra customer info from ERP)

What it holds: customer id (`cid`), birthdate (`bdate`), gender (`gen`). 18,484 rows.

Problem 1: the id has a prefix, so it cannot join to the CRM

How we found it: during integration we saw CRM customers are linked to ERP by `cst_key = cid`. We compared them: CRM key `AW00011000` versus ERP `NASAW00011000`. `WHERE cid LIKE 'NAS%'` returned **11,042 rows** (`LIKE 'NAS%'` means "starts with NAS").

The fix:

```
CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid)) ELSE cid END AS cid
```

If the id starts with `NAS`, take everything from position 4 onward (letters N, A, S occupy positions 1 to 3), giving `AW00011000`. Otherwise leave it. Without this, a LEFT JOIN would match almost no ERP rows and gold would show NULL birthdates for most customers.

Problem 2: impossible birthdates

How we found it: `WHERE bdate < '1924-01-01' OR bdate > GETDATE()`. Result: **16 birthdates in the future** (for example 2050-07-06, 2042-02-22) and **15 before 1924** (for example 1916-02-10; a customer over 100 years old at that data's time).

The fix: `CASE WHEN bdate > GETDATE() THEN NULL ELSE bdate END`. Future dates are impossible, so they become unknown. `GETDATE()` is today's date and time.

Honest gap: the procedure fixes the future dates but does **not** null the 15 very old ones, even though the quality check flags them. They are only suspicious, not provably impossible, so keeping them is a defensible decision, but be ready to explain it.

Problem 3: messy gender values

How we found it: `SELECT DISTINCT gen`. It returned `Male`, `Female`, NULL, `F`, `M`, and values with spaces such as `[M ]`, `[F ]`, `[ ]`, `[  ]`. About 1,472 NULLs plus a handful of blank or single-letter values.

The fix:

```
CASE WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
     WHEN UPPER(TRIM(gen)) IN ('M', 'MALE')   THEN 'Male'
     ELSE 'n/a' END AS gen
```

`TRIM` removes the spaces and makes a blank-only value an empty string, which falls to `n/a`. `IN ('F','FEMALE')` accepts either spelling. Result: Male 8,612, Female 8,396, `n/a` 1,476.

D7. Table 5: `erp_loc_a101` (customer country from ERP)

What it holds: customer id (`cid`) and country (`cntry`). 18,484 rows.

Problem 1: the id has dashes

ERP location uses `AW-00011000`, CRM uses `AW00011000`. Every one of the 18,484 rows has a dash.

The fix: `REPLACE(cid, '-', '')` deletes every dash (replace it with nothing).

Problem 2: the same country spelled many ways

How we found it: `SELECT DISTINCT cntry ORDER BY cntry`. The list showed:

Raw value	Rows	Meaning
United States	3,391	fine
USA	2,591	same country
US	1,500	same country
Germany	1,214	fine
DE	566	same country
NULL	332	unknown
blank / spaces	5	unknown

Why it is a problem: a report grouped by country would show "US", "USA" and "United States" as three separate countries, so US sales would be split and understated.

The fix:

```
CASE WHEN TRIM(cntry) = 'DE'              THEN 'Germany'
     WHEN TRIM(cntry) IN ('US', 'USA')    THEN 'United States'
     WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
     ELSE TRIM(cntry) END AS cntry
```

Codes map to full names, empties become `n/a`, everything else is just trimmed. Result after load: United States 7,482 (3,391 + 2,591 + 1,500), Germany 1,780 (1,214 + 566), `n/a` 337.

D8. Table 6: `erp_px_cat_g1v2` (product categories from ERP)

What it holds: category id (`id` like `CO_RF`), category (`cat`), subcategory (`subcat`), and whether maintenance is needed. 37 rows.

How we checked: trimmed-space check on `cat`, `subcat`, `maintenance`, and `SELECT DISTINCT maintenance`. Nothing wrong: no extra spaces, and maintenance is only `Yes` (20 rows) or `No` (17 rows).

The silver query: a plain `SELECT id, cat, subcat, maintenance FROM bronze...`. No cleaning.

Why load it at all? Rule: gold must read only from silver, never from bronze, so every table needs to exist in silver, even if clean. Keeping that rule means that if the source later gets dirty, you add the fix in one place.

Related issue found later: 7 pedal products have `cat_id = CO_PE` which does not exist in this table, so they show NULL category in gold. This is a source-data gap, not something silver can fix.

D9. One-page summary: problem, how found, fix

Table	Problem	Found with	Fix
crm_cust_info	NULL ids, duplicate customers	`GROUP BY id HAVING COUNT(*) > 1 OR id IS NULL`	`WHERE id IS NOT NULL` + `ROW_NUMBER` keep newest
crm_cust_info	Spaces in names	`col != TRIM(col)`	`TRIM`
crm_cust_info	Coded marital status and gender	`SELECT DISTINCT`	`CASE` mapping, `ELSE 'n/a'`
crm_prd_info	Key joins neither to sales nor ERP	Compare key formats across tables	`SUBSTRING` + `REPLACE` to split into `cat_id` and `prd_key`
crm_prd_info	NULL cost, coded product line	`IS NULL`, `DISTINCT`	`ISNULL(...,0)`, `CASE` mapping
crm_prd_info	200 rows with end date before start	`end < start`	`LEAD` minus 1 day
crm_sales_details	Dates as numbers, 19 are `0`	`<= 0 OR LEN != 8`	`CASE` then `CAST` number to text to date, else NULL
crm_sales_details	35 rows where sales is not qty x price	`sales != qty * price`, NULL and negative tests	Recalculate sales and price with `ABS`, `NULLIF`
erp_cust_az12	`NAS` prefix on 11,042 ids	`LIKE 'NAS%'`	`SUBSTRING` from position 4

Table	Problem	Found with	Fix
erp_cust_az12	16 future birthdates	`bdate > GETDATE()`	set to NULL
erp_cust_az12	Messy gender	`DISTINCT`	`CASE` + `UPPER(TRIM())`
erp_loc_a101	Dashes in ids	Compare with CRM key	`REPLACE(cid, '-', '')`
erp_loc_a101	DE, US, USA, blanks	`DISTINCT cntry`	`CASE` mapping
erp_px_cat_g1v2	None found	Space and DISTINCT checks	Straight copy

D10. The three questions an interviewer will ask about silver

1. **"How did you know what was wrong?"** I profiled bronze with SQL: duplicate and NULL key checks with `GROUP BY ... HAVING`, space checks with `col != TRIM(col)`, `SELECT DISTINCT` on code columns, date-order and range checks, and the business rule sales = quantity x price. Every transformation maps to a check that returned rows.
2. **"How do you know your fixes worked?"** I ran the same checks on silver and expected zero rows, and compared row counts between layers (only the customer table shrank, from 18,494 to 18,484, because of the removed junk and duplicates).
3. **"What decisions did you make that a business owner should approve?"** Keeping the newest duplicate, `ABS` on negative prices, cost 0 for missing cost, `n/a` for unknown values, NULL for impossible dates, and not nulling birthdates before 1924. Each one is a judgement call that should be documented and confirmed with the data owner.

Part E: Every silver query, code first, then a detailed explanation

Format for each query: the **code exactly as in your `5_proc_load_silver.sql`**, then **what it does step by step**, a **worked example with real rows**, and **why it was created**. Comments in the code are yours.

E1. The procedure skeleton (runs around all six queries)

```
CREATE OR ALTER PROCEDURE silver.load_silver AS
BEGIN
    DECLARE @start_time DATETIME, @end_time DATETIME, @batch_start_time DATETIME,
@batch_end_time DATETIME;
    BEGIN TRY
        SET @batch_start_time = GETDATE();
        PRINT 'Loading Silver Layer';
        ...six table loads...
        SET @batch_end_time = GETDATE();
        PRINT '   - Total Load Duration: ' + CAST(DATEDIFF(SECOND, @batch_start_time,
@batch_end_time) AS NVARCHAR) + ' seconds';
    END TRY
```

```
    BEGIN CATCH
        PRINT 'ERROR OCCURED DURING LOADING BRONZE LAYER'
        PRINT 'Error Message' + ERROR_MESSAGE();
        PRINT 'Error Message' + CAST (ERROR_NUMBER() AS NVARCHAR);
        PRINT 'Error Message' + CAST (ERROR_STATE() AS NVARCHAR);
    END CATCH
END
```

How it works

- `CREATE OR ALTER PROCEDURE` saves all the SQL under one name. You run everything with one command: `EXEC silver.load_silver;`. Without it you would paste six long scripts by hand every day.
- `DECLARE @start_time ...` creates four variables (temporary named boxes). A variable named with `@` holds a value while the procedure runs. These hold timestamps.
- `SET @start_time = GETDATE();` stores the current time before a table loads; `SET @end_time = GETDATE();` after. `DATEDIFF(SECOND, @start_time, @end_time)` subtracts them to give seconds taken. `CAST(... AS NVARCHAR)` converts that number to text because `PRINT` and `+` text joining need text.
- `PRINT` writes a message to the output window so you can watch progress.
- `BEGIN TRY ... END TRY` means "run this block, and if any statement fails, stop and jump to CATCH". `ERROR_MESSAGE()`, `ERROR_NUMBER()`, `ERROR_STATE()` return the reason for the failure.

Why it was created: loading must be repeatable, monitored and fail visibly. Timing tells you which table is slow. The CATCH block stops a failure from showing a cryptic crash.

Known flaws: CATCH says BRONZE (copy-paste), all three labels say "Error Message", and the error is only printed, never re-raised, so a scheduler would think the run succeeded. Add `THROW;` at the end of CATCH.

The pattern repeated inside TRY for each table:

```
SET @start_time = GETDATE();
TRUNCATE TABLE silver.xxx;
INSERT INTO silver.xxx (col1, col2, ...)
SELECT ... FROM bronze.xxx;
SET @end_time = GETDATE();
PRINT '>> Load Duration: ' + ...;
```

- `TRUNCATE TABLE` deletes all rows instantly. This is why running the procedure twice never doubles your data: the table is emptied each time (a **full load**).
- `INSERT INTO silver.xxx (col1, col2...)` lists the columns that receive data. Values from the SELECT are matched **by position**, so the SELECT's order must match this list. `dwh_create_date` is deliberately missing from every list, so SQL Server fills it from its `DEFAULT GETDATE()` rule.
- `SELECT ... FROM bronze.xxx` is the real work: read the raw table, clean each column on the way out.

One rule that confuses beginners: inside one SELECT, an expression can only use the **original bronze columns**, never the aliases you create in the same SELECT. So when a later column's formula says `sls_sales`, it means the raw bronze value, not the corrected one. This matters in the sales query below.

E2. Query 1: `silver.crm_cust_info`

```
INSERT INTO silver.crm_cust_info (
    cst_id, cst_key, cst_firstname, cst_lastname,
    cst_marital_status, cst_gndr, cst_create_date
)
SELECT
    cst_id,
    cst_key,
    TRIM(cst_firstname) AS cst_firstname,
    TRIM(cst_lastname) AS cst_lastname,
    CASE
        WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
        WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
        ELSE 'n/a'
    END AS cst_marital_status,
    CASE
        WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
        WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
        ELSE 'n/a'
    END AS cst_gndr,
    cst_create_date
FROM (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) AS
flag_last
    FROM bronze.crm_cust_info
    WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1;
```

Read it from the inside out (SQL runs the innermost query first):

1. **Innermost** `SELECT *, ROW_NUMBER() ... FROM bronze.crm_cust_info WHERE cst_id IS NOT NULL`. First the `WHERE` discards rows whose id is NULL (4 junk rows). Then `ROW_NUMBER()` adds a new column `flag_last`.
    - `PARTITION BY cst_id`: restart numbering for each customer id.
    - `ORDER BY cst_create_date DESC`: inside a customer, the newest created row is first.
2. **Outer** `SELECT ... FROM ( ... ) t WHERE flag_last = 1`: keep only the row numbered 1 per customer, so exactly one row per customer. The inner query must be wrapped as `t` because SQL does not allow filtering on a window function in the query that creates it.
3. **Column cleaning** happens in the outer SELECT: `TRIM` on names, `CASE` on the codes.

Worked example, customer 29466 (three rows in bronze):

created	name	marital	gender	flag_last
2026-01-27	Lance Jimenez	M	M	**1** (kept)
2026-01-26	Lance Jimenez	M	empty	2 (dropped)

created	name	marital	gender	flag_last
2026-01-25	empty	empty	empty	3 (dropped)

Silver result: one row, `Lance`, `Jimenez`, `Married`, `Male`.

How the CASE reads rows: it tests top to bottom and stops at the first match. For `cst_marital_status = ' m'`: `TRIM` gives `'m'`, `UPPER` gives `'M'`, the second WHEN matches, result `Married`. For NULL: neither WHEN matches (NULL is never equal to anything), so `ELSE 'n/a'`.

Why it was created: the CRM stores several versions of a customer; reports need one trusted row each. Duplicated ids would double sales on joins. Codes `M`/`F`/`S` are meaningless to business users, and stray spaces make equal values look different.

E3. Query 2: `silver.crm_prd_info`

```
INSERT INTO silver.crm_prd_info (
    prd_id, cat_id, prd_key, prd_nm, prd_cost, prd_line, prd_start_dt, prd_end_dt
)
SELECT
    prd_id,
    REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,
    SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key,
    prd_nm,
    ISNULL(prd_cost, 0) AS prd_cost,
    CASE
        WHEN UPPER(TRIM(prd_line)) = 'M' THEN 'Mountain'
        WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
        WHEN UPPER(TRIM(prd_line)) = 'S' THEN 'Other Sales'
        WHEN UPPER(TRIM(prd_line)) = 'T' THEN 'Touring'
        ELSE 'n/a'
    END AS prd_line,
    CAST(prd_start_dt AS DATE) AS prd_start_dt,
    CAST(
        LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt) - 1
        AS DATE
    ) AS prd_end_dt
FROM bronze.crm_prd_info;
```

Column by column:

- `prd_id`: copied.
- **`cat_id`:** the raw key is `CO-RF-FR-R92B-58`. `SUBSTRING(prd_key, 1, 5)` takes 5 characters from position 1: `CO-RF`. `REPLACE(..., '-', '_')` converts to `CO_RF`, the exact format of the ERP category table's `id`. This is a **new derived column** so products can join to categories.
- **`prd_key`:** `SUBSTRING(prd_key, 7, LEN(prd_key))` starts at position 7 (position 6 is the dash) and runs to the end: `FR-R92B-58`. This is the form sales uses in `sls_prd_key`, so products can join to sales. The third argument only needs to be large enough; `LEN(prd_key)` is just a safe "to the end".
- `prd_nm`: copied.

- **`prd_cost`:** `ISNULL(prd_cost, 0)` replaces NULL with 0.
- **`prd_line`:** code to word (`R` becomes Road, and so on). `TRIM` is essential because the raw values are like `'R '`.
- **`prd_start_dt`:** `CAST(... AS DATE)` removes the unused time part from bronze's DATETIME.
- **`prd_end_dt`:** the cleverest line. Trace for the Red Sport-100 Helmet (`AC-HE-HL-U509-R`), three versions:

start	LEAD(start) = next row's start	minus 1 day	end in silver	source end (wrong)
2011-07-01	2012-07-01	2012-06-30	2012-06-30	2007-12-28
2012-07-01	2013-07-01	2013-06-30	2013-06-30	2008-12-27
2013-07-01	none (last row)	NULL	NULL = current	empty

- `PARTITION BY prd_key` keeps different products from mixing (each product's versions are compared only with each other).
- `ORDER BY prd_start_dt` makes "next row" mean "next version in time".
- `- 1` works because `prd_start_dt` in bronze is a DATETIME; subtracting 1 from a DATETIME means one day earlier. (It would be an error on a DATE type, which is one reason the cast to DATE is done after.)
- The window uses the original full key (`AC-HE-HL-U509-R`) because the alias `prd_key` created in this SELECT is not visible to the window. That is fine; the full key is unique per product.

Why it was created: to make the key joinable in two directions, make cost and line usable, and replace broken history dates with correct ones. 200 of 397 source rows had an end before the start. The NULL on the latest version is what lets gold select current products with `WHERE prd_end_dt IS NULL`.

E4. Query 3: `silver.crm_sales_details`

```
INSERT INTO silver.crm_sales_details (
    sls_ord_num, sls_prd_key, sls_cust_id, sls_order_dt, sls_ship_dt, sls_due_dt,
    sls_sales, sls_quantity, sls_price
)
SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    CASE
        WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_order_dt AS VARCHAR) AS DATE)
    END AS sls_order_dt,
    CASE
        WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_ship_dt AS VARCHAR) AS DATE)
    END AS sls_ship_dt,
    CASE
        WHEN sls_due_dt = 0 OR LEN(sls_due_dt) != 8 THEN NULL
```

```
        ELSE CAST(CAST(sls_due_dt AS VARCHAR) AS DATE)
    END AS sls_due_dt,
    CASE
        WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity *
ABS(sls_price)
            THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
    END AS sls_sales,
    sls_quantity,
    CASE
        WHEN sls_price IS NULL OR sls_price <= 0
            THEN sls_sales / NULLIF(sls_quantity, 0)
        ELSE sls_price
    END AS sls_price
FROM bronze.crm_sales_details;
```

Copied unchanged: order number, product key, customer id, quantity.

The three date columns (same logic three times):

- Bronze holds 11 May 2013 as the integer `20130511`.
- `LEN(sls_order_dt) != 8`: SQL Server turns the integer into text and counts characters. A real date has 8 digits. `sls_order_dt = 0` catches the placeholder zeros.
- If invalid, the result is **NULL** (unknown) instead of a made-up date.
- Otherwise `CAST(sls_order_dt AS VARCHAR)` makes the text `'20130511'`, and the outer `CAST(... AS DATE)` reads the text in `yyyymmdd` format as the date 2013-05-11. Two casts are needed because SQL Server will not turn an integer directly into a date.
- Verified: 19 order dates were `0` (for example order `SO64338`). They become NULL; the sales amounts stay.

`sls_sales` (rule: Sales = Quantity x Price): replace sales with `quantity x ABS(price)` if sales is NULL, zero or negative, or does not equal `quantity x ABS(price)`. Otherwise keep it. `ABS` is absolute value (removes a minus sign).

`sls_price`: if price is NULL, zero or negative, derive it as `sls_sales / NULLIF(sls_quantity, 0)`. `NULLIF(qty, 0)` turns a zero quantity into NULL so the division gives NULL rather than a crash.

Worked examples with real rows (remember: formulas read the original bronze values):

Order	Raw (sales, qty, price)	Sales result	Price result	Explanation
SO57804	769, 1, -769	769	769	price is negative; sales already equals 1 x ABS(-769), so kept; price derived as 769 / 1
SO51259	10, 2, NULL	10	5	`10 != 2 * ABS(NULL)` evaluates to unknown, not true, so sales is kept; price derived as 10 / 2

Order	Raw (sales, qty, price)	Sales result	Price result	Explanation
SO58335	35, 2, 35	**70**	35	35 does not equal 2 x 35, so sales is recalculated to 70

Why it was created: sales is the revenue table; one wrong row corrupts totals. The dates must be real dates for time analysis. 35 rows broke the rule. **Assumption to declare:** `ABS` treats negative prices as sign typos; if the business records refunds as negatives, this logic is wrong.

E5. Query 4: `silver.erp_cust_az12`

```
INSERT INTO silver.erp_cust_az12 (cid, bdate, gen)
SELECT
    CASE
        WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
        ELSE cid
    END AS cid,
    CASE
        WHEN bdate > GETDATE() THEN NULL
        ELSE bdate
    END AS bdate,
    CASE
        WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
        WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
        ELSE 'n/a'
    END AS gen
FROM bronze.erp_cust_az12;
```

- **`cid`:** `LIKE 'NAS%'` means "starts with NAS" (`%` is any characters). Position 1 to 3 are N, A, S, so `SUBSTRING(cid, 4, LEN(cid))` starts at position 4 and keeps the rest. `NASAW00011000` becomes `AW00011000`, identical to the CRM `cst_key`. Ids that don't start with NAS pass through unchanged (about 7,442 of the 18,484).
- **`bdate`:** a birthdate later than now is impossible, so it becomes NULL (16 rows, e.g. 2050-07-06). Dates earlier than 1924 are flagged by the quality check but not changed by this query.
- **`gen`:** `UPPER(TRIM(gen))` normalises case and spaces. `IN ('F','FEMALE')` is shorthand for `= 'F' OR = 'FEMALE'`. A blank, a lone space or NULL matches nothing, so it falls to `n/a`.

Why it was created: without the id fix, most ERP rows would never match a CRM customer and gold would show NULL birthdates. Impossible dates and inconsistent gender spellings would pollute age and demographic analysis.

E6. Query 5: `silver.erp_loc_a101`

```
INSERT INTO silver.erp_loc_a101 (cid, cntry)
SELECT
```

```
    REPLACE(cid, '-', '') AS cid,
    CASE
        WHEN TRIM(cntry) = 'DE' THEN 'Germany'
        WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
        WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'n/a'
        ELSE TRIM(cntry)
    END AS cntry
FROM bronze.erp_loc_a101;
```

- **`cid`:** `REPLACE(cid, '-', '')` replaces every dash with nothing: `AW-00011000` becomes `AW00011000`, matching CRM.
- **`cntry`:** tested top to bottom, first match wins. `DE` becomes Germany; `US` or `USA` becomes United States; an empty string or NULL becomes `n/a`; everything else (Australia, France...) is returned trimmed.
- Order matters only in that `ELSE` is last. The NULL case must be tested with `IS NULL`, because `NULL = ''` is never true.

Results: United States 7,482 (3,391 + 2,591 + 1,500), Germany 1,780 (1,214 + 566), `n/a` 337.

Why it was created: to join to CRM, and so grouping by country counts each country once. Before the fix, US sales would be spread across `US`, `USA` and `United States`, understating the US total.

E7. Query 6: `silver.erp_px_cat_g1v2`

```
INSERT INTO silver.erp_px_cat_g1v2 (id, cat, subcat, maintenance)
SELECT id, cat, subcat, maintenance
FROM bronze.erp_px_cat_g1v2;
```

No transformation: the checks found no spaces and `maintenance` is only `Yes` or `No`. It exists in silver because **gold reads only from silver**, and because if the source later becomes dirty you add the fix here in one place. Its `id` (like `CO_RF`) is the value the product `cat_id` joins to. Seven pedal products (`CO_PE`) have no row here, which is why their category is NULL in gold.

E8. After the procedure: what proves it worked

Run `quality_checks_silver.sql`. Every query that is supposed to return nothing should return nothing:

- no duplicate or NULL `cst_id` or `prd_id`
- no names with leftover spaces
- no `prd_end_dt < prd_start_dt`
- no `sls_order_dt` later than ship or due date
- no row where sales differs from quantity x price, or any of them is NULL or not positive
- `DISTINCT` on gender, marital status, product line, country, maintenance shows only the clean values

Note: the due-date check in that file reads `bronze.crm_sales_details`, because after conversion to DATE the integer tests no longer apply to silver. And two silver outcomes are intentional, not errors: the 19 NULL order dates and the 16 NULL birthdates.

E9. One-line memory aid for each query

1. **Customers:** drop empty IDs, keep newest duplicate, trim spaces, decode M/F/S.
2. **Products:** split key into category and product, decode line, default cost, rebuild end dates with LEAD.
3. **Sales:** text-to-date for number dates, repair sales and price so Sales = Quantity x Price.
4. **ERP customers:** strip NAS from id, null future birthdates, standardise gender.
5. **ERP locations:** strip dashes, standardise country names.
6. **ERP categories:** copy, already clean.

Part F: Gold layer queries, joins, and how to build dimension and fact tables

Important note: your gold view scripts were not uploaded, so the three views below are my reconstruction from your data catalog, data model image and quality checks. They follow the standard pattern for this project. Compare them with your own scripts and tell me about any difference. Every number was verified by running this logic on your CSVs (in DuckDB, not SQL Server).

Same format as Part E: code first, then a detailed explanation.

F1. What gold is for and the vocabulary you need

Silver is clean, but it is still **shaped like the source systems**: one customer is spread over three tables, one product over two, and sales uses business keys. An analyst would need to know all of that to answer "sales by country and category". Gold removes that burden by reshaping the data into a **star schema**: one central fact table surrounded by dimension tables.

Term	Meaning	Example here
Fact table	A table of **events or measurements**: numbers you add up or count. Many rows. Contains keys to dimensions plus measures.	`fact_sales`: quantity, price, sales_amount
Dimension table	A table that **describes** the things involved in the events: who, what, where, when. Fewer rows, many descriptive columns.	`dim_customers`, `dim_products`
Grain	What one row of the table represents. Decide this first.	fact_sales: one row per source sales line; dim_customers: one row per customer;

Term	Meaning	Example here
		dim_products: one row per current product
Natural / business key	The key the source system uses	`cst_id`, `prd_key`
Surrogate key	A key **generated by the warehouse**, meaningless outside it, used to link fact to dimension	`customer_key`, `product_key`
Star schema	Fact in the middle, dimensions around it, joined by surrogate keys	your data model image

Why gold objects are views: the data is already clean and small, so a view (a saved SELECT) is enough. It stores no data, needs no load procedure, and always shows whatever is currently in silver. Trade-off: the query runs each time someone reads the view. For big data you would materialise it into real tables.

How a view is created (the standard re-runnable pattern):

```
IF OBJECT_ID('gold.dim_customers', 'V') IS NOT NULL
    DROP VIEW gold.dim_customers;
GO
CREATE VIEW gold.dim_customers AS
SELECT ... ;
GO
```

`OBJECT_ID(name, 'V')` returns the object id if a **V**iew with that name exists; if so we drop it first so the script can be run again without an error. `GO` ends the batch (a `CREATE VIEW` must be the only statement in its batch).

F2. Why joins at all, and why this particular join type

Why we need joins

Because the information about one business object lives in several tables:

- A **customer** is in `crm_cust_info` (name, marital status, gender), `erp_cust_az12` (birthdate, gender) and `erp_loc_a101` (country). A join is the only way to put them into one row.
- A **product** is in `crm_prd_info` (name, cost, line) and `erp_px_cat_g1v2` (category, subcategory, maintenance).
- A **sale** only contains the source's customer id and product key. To link it to a dimension row we **look up** the dimension's surrogate key with a join.

A join glues rows from two tables together **where a key column matches** (`ON a.key = b.key`).

The join types, with a tiny example

Imagine a customers table (ids 1, 2, 3) and a location table with rows for ids 1, 2 and 9.

Join	Result	When it is right
INNER JOIN	Only ids that exist in **both**: 1, 2	When a row without a match is truly useless and should disappear
LEFT JOIN	**All rows from the left table**, plus match data where it exists: 1, 2, 3 (3 gets NULLs for location)	When the left table is the **master list** and you must not lose any of its rows
RIGHT JOIN	All rows from the right table: 1, 2, 9	Same as LEFT with the tables swapped; rarely used because it reads backwards
FULL OUTER JOIN	Everything from both: 1, 2, 3, 9	Comparing two sources to find mismatches (reconciliation)
CROSS JOIN	Every row paired with every row (3 x 3 = 9)	Generating combinations (for example a calendar); almost never for a model

Why LEFT JOIN is the right choice here

The deciding question is: **which table is the master list, and what would losing a row cost?**

1. **`dim_customers`:** the CRM customer table is the master (it holds the customer id used by sales). ERP tables only add attributes. So `crm_cust_info` is the left table and the two ERP tables are LEFT JOINed. If an ERP row is missing, we still want the customer, just with a NULL birthdate or country. INNER JOIN would silently delete that customer, and every sale they made would later lose its customer link.
2. **`dim_products`:** `crm_prd_info` is the master; the ERP category table only adds category text. LEFT JOIN keeps a product even if its category is missing.
3. **`fact_sales`:** the sales table is the master (it is the thing we measure). LEFT JOIN to both dimensions keeps **every sale**. If a lookup fails, the key is NULL and the quality check catches it. INNER JOIN would silently drop that sale and **understate revenue** with no warning.
4. **Why not RIGHT or FULL?** They would add rows from the secondary tables that have no master row. For example a RIGHT JOIN from fact to dim_customers would add one fact row for every customer who never bought anything, with NULL sales. That changes the **grain** of the fact table and corrupts counts. FULL JOIN has the same problem. They are tools for reconciliation, not for building a model.
5. **Why not CROSS JOIN?** It multiplies rows (every customer paired with every product) and has no business meaning here.

Verified: what each choice does to your data

Experiment on your data	Result
`dim_customers` with LEFT JOINs	18,484 rows
`dim_customers` with INNER JOINs	18,484 rows (same, today)

Experiment on your data	Result
`dim_products` (current versions) with LEFT JOIN to category	**295** rows
`dim_products` with INNER JOIN to category	**288** rows. 7 pedal products with category id `CO_PE` disappear
`fact_sales` with LEFT JOINs	60,398 rows
`fact_sales` with INNER JOINs	60,398 rows (same, today)
`fact_sales` if the product dimension kept **all versions**	**89,818** rows, which is 29,420 extra duplicated rows
ERP customers with no CRM match (what a RIGHT/FULL join would add)	0

Be honest about what this shows: on your current data, INNER and LEFT give the same counts for the customer dimension and the fact table, because every key happens to match. The one place they differ is `dim_products` (7 products). So the choice of LEFT JOIN is about **safety and correct meaning**, not about today's numbers: tomorrow's file may contain an unmatched row, and LEFT JOIN means the problem shows up as a visible NULL rather than as silently missing data. That is exactly what to say in an interview.

The last row of the table matters even more: a join on a **non-unique key multiplies rows**. If the product dimension contained all three versions of a product, each sale would match three rows, and sales totals would be inflated. That is why `dim_products` keeps only current versions and why we check key uniqueness afterwards.

The golden rule of dimension joins: the key you join on must be **unique on the dimension side**. Joining many-to-one (many sales to one product) never changes the row count of the fact. Joining many-to-many multiplies rows.

F3. Exploration before building `dim_customers` (how we decide the logic)

Before writing the view you run exploratory queries on silver. This is how you discover the rules.

Query 1: join the three customer tables and check nothing multiplied

```
SELECT ci.cst_id, COUNT(*)
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid
GROUP BY ci.cst_id
HAVING COUNT(*) > 1;
```

If the join keys are unique, each customer matches at most one row in each ERP table, and this returns **nothing**. If it returns rows, the join created duplicates. Verified: nothing returned, and the join produces

18,484 rows, the same as the customer count. (This is why silver had to make `cid` joinable and unique first.)

Query 2: the gender conflict check (customer gender exists in both CRM and ERP)

```
SELECT DISTINCT
    ci.cst_gndr,
    ca.gen,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr
         ELSE COALESCE(ca.gen, 'n/a') END AS new_gen
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;
```

`DISTINCT` lists each combination once. The counts behind it:

CRM gender	ERP gender	Customers	Meaning
Male	Male	6,298	agree
Female	Female	6,099	agree
n/a	Female	2,280	CRM unknown, ERP knows
n/a	Male	2,274	CRM unknown, ERP knows
Male	n/a	752	only CRM knows
Female	n/a	709	only CRM knows
Female	Male	40	**conflict**
Male	Female	17	**conflict**
n/a	n/a	15	nobody knows

Decision: **CRM is the master**; when CRM says `n/a` take the ERP value; otherwise `n/a`. That fills 4,554 customers (2,280 + 2,274) and leaves 15 unknown. The 57 conflicts are resolved in favour of CRM, a business rule that must be documented and agreed with the data owner.

F4. `gold.dim_customers`

```
CREATE VIEW gold.dim_customers AS
SELECT
    ROW_NUMBER() OVER (ORDER BY ci.cst_id) AS customer_key,
    ci.cst_id            AS customer_id,
    ci.cst_key           AS customer_number,
    ci.cst_firstname     AS first_name,
    ci.cst_lastname      AS last_name,
    la.cntry             AS country,
    ci.cst_marital_status AS marital_status,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr
         ELSE COALESCE(ca.gen, 'n/a')
    END                  AS gender,
```

```
    ca.bdate             AS birthdate,
    ci.cst_create_date   AS create_date
FROM silver.crm_cust_info ci
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid;
```

Step by step

1. **`FROM silver.crm_cust_info ci`:** the master list. One row per customer. `ci` is a short alias for the table name.
2. **`LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid`:** attach birthdate and ERP gender. The match is on the customer number (`AW00011000`), which is only possible because silver stripped `NAS` from `cid`.
3. **`LEFT JOIN silver.erp_loc_a101 la ON ci.cst_key = la.cid`:** attach country. This works because silver removed the dashes.
4. **Gender `CASE`:** CRM first, ERP as fallback. `COALESCE(a, b)` returns the first value that is not NULL. It matters here because if the LEFT JOIN found no ERP row, `ca.gen` would be NULL, and we want `n/a` instead.
5. **`ROW_NUMBER() OVER (ORDER BY ci.cst_id) AS customer_key`:** creates the **surrogate key**: 1, 2, 3 ... in order of customer id. It is deterministic here because `cst_id` is unique.
6. **Aliases (`AS first_name` ...):** business-friendly names, as the naming convention requires for gold. It also **selects only useful columns**: `cst_gndr` and `gen` are merged into one `gender` column; the technical column `dwh_create_date` is not exposed.

Result: 18,484 rows; gender Male 9,341, Female 9,128, n/a 15; sample row: key 1, customer 11000, `AW00011000`, Jon Yang, Australia, Married, Male, born 1971-10-06.

Why integrate three tables into one dimension? Analysts want one customer entity with all attributes. They should not need to know which source holds which column.

Why a surrogate key at all? (a) Source keys can change, repeat across systems or be text; (b) an integer join is fast; (c) it decouples the warehouse from the source. Caveat: `ROW_NUMBER()` keys are **recalculated every time the view runs**. If a customer is added in the middle of the id order, keys of later customers shift. A physical table with an `IDENTITY` column, or a hash of the business key, never changes existing keys. Be ready to mention this.

F5. `gold.dim_products`

```
CREATE VIEW gold.dim_products AS
SELECT
    ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,
    pn.prd_id       AS product_id,
    pn.prd_key      AS product_number,
    pn.prd_nm       AS product_name,
    pn.cat_id       AS category_id,
    pc.cat          AS category,
    pc.subcat       AS subcategory,
    pc.maintenance  AS maintenance,
    pn.prd_cost     AS cost,
```

```
    pn.prd_line     AS product_line,
    pn.prd_start_dt AS start_date
FROM silver.crm_prd_info pn
LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id
WHERE pn.prd_end_dt IS NULL;
```

Step by step

1. **`FROM silver.crm_prd_info pn`:** master list of products and their versions.
2. **`WHERE pn.prd_end_dt IS NULL`:** keep only the **current version** of each product (397 version rows become 295 products). This is the requirement "latest data only, no historisation". It also prevents the explosion shown earlier: with all versions, the fact table would have 89,818 rows instead of 60,398. The silver `LEAD` logic is what made `prd_end_dt IS NULL` mean "current".
3. **`LEFT JOIN silver.erp_px_cat_g1v2 pc ON pn.cat_id = pc.id`:** add category, subcategory and maintenance. `cat_id` exists only because silver derived it from the product key. `IS NULL` is used (not `= NULL`) because nothing ever equals NULL.
4. **Surrogate key:** ordered by start date then product number so it is deterministic.
5. **Aliases and column order:** identifiers first, then descriptive attributes, then measures-like fields (cost), then dates, matching the catalog.

Why LEFT, again: INNER JOIN gives **288** products, silently losing the 7 pedal products whose category id `CO_PE` is missing from the ERP table. LEFT JOIN keeps all 295, with a NULL category for those 7. (No sales exist for them yet, but a future sale would be orphaned under INNER JOIN.) Hiding a source data gap is worse than showing it.

F6. `gold.fact_sales`

```
CREATE VIEW gold.fact_sales AS
SELECT
    sd.sls_ord_num  AS order_number,
    pr.product_key  AS product_key,
    cu.customer_key AS customer_key,
    sd.sls_order_dt AS order_date,
    sd.sls_ship_dt  AS shipping_date,
    sd.sls_due_dt   AS due_date,
    sd.sls_sales    AS sales_amount,
    sd.sls_quantity AS quantity,
    sd.sls_price    AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products  pr ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id;
```

Step by step

1. **`FROM silver.crm_sales_details sd`:** the driving table. **Grain: one row per source sales line**. We never want fewer or more rows than silver has: 60,398.
2. **`LEFT JOIN gold.dim_products pr ON sd.sls_prd_key = pr.product_number`:** this is a **dimension lookup**. The sale carries the product's **business key**; we look up the dimension row with that business key and take its **surrogate key** `product_key`. The join key is `product_number`

because that is the silver product key without the category prefix (`FR-R92B-58`), which is what sales stores.

3. **`LEFT JOIN gold.dim_customers cu ON sd.sls_cust_id = cu.customer_id`:** same idea: sales stores `cst_id`; the lookup returns `customer_key`.
4. **Why join to the gold dimensions and not to the silver tables?** The surrogate keys only exist in gold. The fact must hold the same keys the dimensions publish, otherwise the star schema's relationships would not line up.
5. **Select list:** replace the two business keys with the two surrogate keys, keep the order number as a degenerate identifier, then dates, then **measures** (`sales_amount`, `quantity`, `price`).
6. **Column order convention:** keys first, dates next, measures last.

Result: 60,398 rows (same as silver), 0 rows with a missing product or customer key, 19 rows with NULL `order_date` (from silver).

Why LEFT JOIN, and not INNER: both give 60,398 today, but they behave differently when a key does not match. LEFT keeps the sale with a NULL key, where the gold quality check will find it. INNER would remove the sale, so total revenue would quietly drop with no error. In reporting, missing revenue is the worst kind of bug because nobody sees it.

F7. Quality checks on gold (`quality_checks_gold.sql`)

Check 1: surrogate key uniqueness

```
SELECT customer_key, COUNT(*) AS duplicate_count
FROM gold.dim_customers
GROUP BY customer_key
HAVING COUNT(*) > 1;
```

Bucket rows by key and keep only buckets with more than one row. A primary key must be unique, so the correct result is **empty**. The same check is done for `product_key`. It protects against the join fan-out described above.

Check 2: referential integrity (does the star connect?)

```
SELECT *
FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c ON c.customer_key = f.customer_key
LEFT JOIN gold.dim_products  p ON p.product_key  = f.product_key
WHERE p.product_key IS NULL OR c.customer_key IS NULL;
```

Left-join the fact to each dimension on the surrogate keys and look for rows where the dimension side is NULL. Any row returned is a sale that cannot be tied to a customer or product. Correct result: **empty** (verified 0 rows). Note this deliberately uses LEFT JOIN: with INNER JOIN the bad rows would vanish and the check would always look clean.

F8. The recipe: how to create a dimension and a fact table for any project

To create a dimension

1. **Identify the business object** (customer, product, store, date).
2. **Find every silver table that describes it** and pick the **master** table (the one whose key the facts use).
3. **Define the grain:** one row per what? (one per customer; one per current product).
4. **LEFT JOIN** the other tables onto the master using keys that silver made joinable.
5. **Check the join did not duplicate rows** (`GROUP BY key HAVING COUNT(*) > 1`).
6. **Resolve overlapping columns** (gender from CRM and ERP) with explicit rules and `COALESCE`/`CASE`.
7. **Filter to the version you need** (current only, or keep history with an SCD design).
8. **Generate the surrogate key** (`ROW_NUMBER()` in a view, `IDENTITY` in a table).
9. **Rename columns** to business-friendly names; **drop technical columns**; order them logically.
10. **Validate** key uniqueness.

To create a fact table

1. **Choose the business process** (a sale) and its **grain** (one row per order line).
2. **Use the transaction table** from silver as the driving (left) table.
3. **Look up each dimension's surrogate key** with a LEFT JOIN on the business key; select the surrogate key and drop the business key.
4. **Keep the measures** (numbers to sum: amount, quantity, price) and the dates.
5. **Validate:** row count equals silver, and no NULL dimension keys (the referential integrity check).

Fact versus dimension, quick test: if you would put it in `SUM()` or `COUNT()`, it is a measure (fact). If you would put it in `GROUP BY` or a filter (`country`, `category`), it is a dimension attribute.

Why a star schema pays off: the question "sales by country and category" becomes:

```
SELECT c.country, p.category, SUM(f.sales_amount) AS total_sales
FROM gold.fact_sales f
JOIN gold.dim_customers c ON c.customer_key = f.customer_key
JOIN gold.dim_products  p ON p.product_key  = f.product_key
GROUP BY c.country, p.category;
```

Two simple joins, no cleaning logic, no knowledge of CRM versus ERP. (Here a plain `JOIN` (INNER) is fine because you are querying, not building; you only want rows that have both sides.) Verified on your data: Bikes about 28.3M, Accessories about 0.70M, Clothing about 0.34M of total sales amount, and United States about 9.16M, Australia about 9.06M by country.

F9. Cross-questions on gold and joins, with answers

1. **Why LEFT JOIN instead of INNER JOIN?** LEFT keeps every row of the master table and exposes missing matches as NULLs that checks can find. INNER silently discards rows and can understate revenue. Today they return equal counts for customers and the fact, but `dim_products` shows the difference (295 versus 288).
2. **Why not RIGHT or FULL JOIN?** They add rows from the secondary table that have no master row, which changes the grain and corrupts counts. They are for reconciling two sources.

3. **In `fact_sales`, why join on business keys and not surrogate keys?** The silver sales table only contains business keys; the surrogate keys are created in gold. The join is the lookup that turns one into the other.
4. **Why join the fact to gold dimensions rather than silver tables?** Because the surrogate keys exist only in gold dimensions; the fact has to use the same ones.
5. **What happens if the join key is not unique in the dimension?** Rows multiply. Example: with all product versions, the fact would have 89,818 rows instead of 60,398, and totals would inflate.
6. **How do you guarantee uniqueness?** Filter to current versions, enforce unique keys in silver, and run the duplicate check on the surrogate keys.
7. **Which table is the master in `dim_customers` and why?** `crm_cust_info`, because sales references its customer id, and CRM is the system of record. ERP tables only enrich.
8. **How did you resolve gender in two systems?** CRM first; if CRM is `n/a` use ERP; else `n/a`. 4,554 customers recovered, 15 left unknown, 57 conflicts resolved in favour of CRM. It is a business rule to confirm with the owner.
9. **Why is `dim_products` only 295 rows when silver has 397?** Silver keeps history; gold keeps only current versions because the requirement is latest data only. For history you would build a slowly changing dimension type 2 and link facts to the version valid on the order date.
10. **Why views and not tables?** No storage, no load job, always current; acceptable for small data. For large data, materialise.
11. **Are `ROW_NUMBER()` surrogate keys safe?** Not completely: they can shift when data changes. An `IDENTITY` column or hashed business key is more stable.
12. **What is the grain of `fact_sales`?** One row per sales line in the source: 60,398 rows.
13. **Why does the referential integrity check use LEFT JOIN?** Because INNER JOIN would hide orphan rows and the check would always pass.
14. **What did the quality checks find?** Zero duplicate keys and zero orphans; the real data issues were found earlier: 7 products without a category and 19 orders with no order date.

Part G: The test cases (quality checks) for silver and gold

The two files are `quality_checks_silver.sql` and `quality_checks_gold.sql`. Every result below was obtained by running the same logic on your CSVs (DuckDB, not SQL Server).

G1. What a "test case" means in this project

There is no testing framework here. A test case is simply a **SQL query written so that it returns rows only when something is wrong**. You run it, and:

- **Expectation: No Results** means an empty result is a **pass**. Any row returned is a **failure** and is a concrete example of the problem.
- Some checks are **eyeball checks** (`SELECT DISTINCT ...`). They return rows by design, and you read the list to see whether the values are clean (for example only `Male`, `Female`, `n/a`).

You use the same checks twice:

1. **Before building a layer**, on the raw data, to *discover* what is wrong (how we found the problems).
2. **After loading a layer**, to *prove* the fixes worked.

A test that has never failed is unproven. That is why several checks below also show what they return on **bronze** (where they find problems) next to what they return on **silver** (where they should find none).

When they run: manually, after `EXEC silver.load_silver;` and after the gold views are created. The file's header says: "Run these checks after data loading Silver Layer. Investigate and resolve any discrepancies found."

G2. Silver layer test cases, one by one

Table `silver.crm_cust_info`

Test S1: NULL or duplicate primary key

```
SELECT cst_id, COUNT(*)
FROM silver.crm_cust_info
GROUP BY cst_id
HAVING COUNT(*) > 1 OR cst_id IS NULL;
```

- **How it works:** `GROUP BY cst_id` puts rows with the same id in one bucket; `COUNT(*)` counts the bucket; `HAVING COUNT(*) > 1` keeps buckets with more than one row; `OR cst_id IS NULL` also catches rows with no id.
- **What it proves:** every customer has exactly one row, so joins will not multiply sales.
- **On bronze:** returns rows (4 NULL ids, and ids such as 29466 with 3 rows). **On silver:** empty. **Pass.**
- **Fixed by:** `WHERE cst_id IS NOT NULL` plus `ROW_NUMBER()` in the silver load.

Test S2: unwanted spaces

```
SELECT cst_key
FROM silver.crm_cust_info
WHERE cst_key != TRIM(cst_key);
```

- **How it works:** `TRIM` removes leading and trailing spaces. If the trimmed value differs from the original, the original had spaces, and the row is returned.
- **Note:** the file only tests `cst_key`. The names were the columns that actually had spaces, so the same test pattern is worth running on `cst_firstname` and `cst_lastname`. On bronze that finds **29 customer rows** (17 first names and 22 last names, some rows with both). On silver: 0. **Pass.**
- **Fixed by:** `TRIM()` in the load.

Test S3: standardisation (eyeball check)

```
SELECT DISTINCT cst_marital_status
FROM silver.crm_cust_info;
```

- **How it works:** `DISTINCT` lists every different value once.
- **Expected:** only readable values. **Actual on silver:** `Married`, `Single`. (Gender, if you run the same test: `Male`, `Female`, `n/a`.) **Pass.**

- **Why no `n/a` for marital status:** the 7 NULL values in bronze belonged to rows removed as junk or older duplicates. On bronze this test shows `M`, `S`, NULL, which is the problem it was written to expose.

Table `silver.crm_prd_info`

Test S4: NULL or duplicate `prd_id`

```
SELECT prd_id, COUNT(*)
FROM silver.crm_prd_info
GROUP BY prd_id
HAVING COUNT(*) > 1 OR prd_id IS NULL;
```

Same technique as S1. **Result: empty on bronze and silver. Pass.** Note that `prd_id` is unique even though `prd_key` repeats (a product has several versions, each with its own `prd_id`). That is why the primary key here is `prd_id`, not `prd_key`.

Test S5: spaces in product name

```
SELECT prd_nm FROM silver.crm_prd_info WHERE prd_nm != TRIM(prd_nm);
```

Result: empty. Pass. The silver load does not trim `prd_nm`; this test is what proves it did not need to.

Test S6: NULL or negative cost

```
SELECT prd_cost
FROM silver.crm_prd_info
WHERE prd_cost < 0 OR prd_cost IS NULL;
```

- **On bronze:** 2 rows (NULL cost, the two `HL Road Frame` products); no negatives. **On silver:** empty. **Pass.**
- **Fixed by:** `ISNULL(prd_cost, 0)`.
- **Honest note:** this test passes partly because the fix replaced NULL with 0, so it confirms "no NULLs", not "all costs are real".

Test S7: product line standardisation (eyeball check)

```
SELECT DISTINCT prd_line FROM silver.crm_prd_info;
```

Actual on silver: `Mountain`, `Road`, `Touring`, `Other Sales`, `n/a`. On bronze: `R`, `M`, `S`, `T` (with trailing spaces) and NULL. **Pass.**

Test S8: invalid date order

```
SELECT * FROM silver.crm_prd_info WHERE prd_end_dt < prd_start_dt;
```

- A version cannot end before it starts. **On bronze: 200 rows. On silver: empty. Pass.**
- **Fixed by:** the `LEAD(...) - 1` end-date rebuild.

Table `silver.crm_sales_details`

Test S9: invalid dates stored as numbers

```
SELECT NULLIF(sls_due_dt, 0) AS sls_due_dt
FROM bronze.crm_sales_details
WHERE sls_due_dt <= 0
   OR LEN(sls_due_dt) != 8
```

```
   OR sls_due_dt > 20500101
   OR sls_due_dt < 19000101;
```

- **How it works:** a valid `yyyymmdd` integer is positive, has 8 digits and lies between 1900 and 2050. The `WHERE` returns anything outside that. `NULLIF(x, 0)` turns a `0` into NULL for display.
- **Why it reads from `bronze`:** the silver columns are already DATE, so tests on integer length would be meaningless there. It is a diagnostic on the raw source, and the file's heading ("Checking silver") is slightly misleading.
- **Result for due date: empty**, meaning due dates were fine. If you point the same test at `sls_order_dt` it returns **19 rows** (all `0`). So the query shown in the file did not detect a problem; the order-date version did. In an interview you can say: "I ran this pattern on all three date columns; only the order date had invalid values."
- **Fixed by:** the `CASE ... NULL ... CAST(CAST(...))` conversion. After the load, the 19 order dates are NULL (intentional, not a failure).

Test S10: order date after ship or due date

```
SELECT * FROM silver.crm_sales_details
WHERE sls_order_dt > sls_ship_dt OR sls_order_dt > sls_due_dt;
```

- A product cannot ship before it is ordered. **Result: empty on bronze and silver.** This is a **guard test**: it found nothing to fix but stays in the suite to catch future bad data. (Rows with a NULL date are never returned, because a comparison with NULL is never true.)

Test S11: business-rule consistency (Sales = Quantity x Price)

```
SELECT DISTINCT sls_sales, sls_quantity, sls_price
FROM silver.crm_sales_details
WHERE sls_sales != sls_quantity * sls_price
   OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL
   OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0
ORDER BY sls_sales, sls_quantity, sls_price;
```

- **How it works:** returns every row that breaks the formula, or has a NULL or non-positive value in any of the three columns. `DISTINCT` and `ORDER BY` just make the output compact and readable.
- **On bronze: 35 rows. On silver: empty. Pass.**
- **Fixed by:** recalculating `sls_sales` and `sls_price`.
- This is the most valuable test in the suite, because it protects **revenue**.

Table `silver.erp_cust_az12`

Test S12: out-of-range birthdates

```
SELECT DISTINCT bdate
FROM silver.erp_cust_az12
WHERE bdate < '1924-01-01' OR bdate > GETDATE();
```

- **Expectation in the file:** birthdates between 1924-01-01 and today.
- **On bronze:** 31 rows (15 before 1924, 16 in the future). **On silver: 15 rows are still returned (1916 to 1923). This is the only silver test that does not return empty.**
- **Why:** the silver load nulls **future** birthdates (the 16 are gone), but does not touch the 15 very old ones. So the test correctly reports them.

- **What to say:** either (a) change the load to also NULL birthdates before 1924 so the test is clean, or (b) relax the test to "future dates only" and accept ages over 100 as plausible. Pick one, because a test that is permanently red trains people to ignore it.

Test S13: gender standardisation (eyeball check)

```
SELECT DISTINCT gen FROM silver.erp_cust_az12;
```

Actual on silver: `Male`, `Female`, `n/a`. On bronze: `Male`, `Female`, `F`, `M`, NULL, blanks and values with spaces. **Pass.**

Table `silver.erp_loc_a101`

Test S14: country standardisation (eyeball check)

```
SELECT DISTINCT cntry FROM silver.erp_loc_a101 ORDER BY cntry;
```

Actual on silver: `Australia`, `Canada`, `France`, `Germany`, `United Kingdom`, `United States`, `n/a`. On bronze the list also contained `DE`, `US`, `USA`, NULL and blanks. **Pass.**

Table `silver.erp_px_cat_g1v2`

Test S15: spaces in category columns

```
SELECT * FROM silver.erp_px_cat_g1v2
WHERE cat != TRIM(cat) OR subcat != TRIM(subcat) OR maintenance != TRIM(maintenance);
```

Result: empty on bronze and silver. Pass. The table needed no cleaning, and this test is the evidence.

Test S16: maintenance values (eyeball check)

```
SELECT DISTINCT maintenance FROM silver.erp_px_cat_g1v2;
```

Actual: `Yes`, `No`. **Pass.**

G2 summary: silver test results

#	Table	Test	On bronze	On silver	Status
S1	crm_cust_info	NULL/duplicate id	4 NULL ids + duplicates	empty	Pass
S2	crm_cust_info	Spaces	29 rows (names)	0	Pass
S3	crm_cust_info	Marital values	M, S, NULL	Married, Single	Pass
S4	crm_prd_info	NULL/duplicate id	empty	empty	Pass
S5	crm_prd_info	Spaces in name	empty	empty	Pass
S6	crm_prd_info	NULL/negative cost	2	0	Pass
S7	crm_prd_info	Product line values	R, M, S, T, NULL	readable words	Pass

#	Table	Test	On bronze	On silver	Status
S8	crm_prd_info	End before start	200	0	Pass
S9	crm_sales_details	Invalid dates (bronze)	due: 0; order: 19	n/a (diagnostic)	Pass
S10	crm_sales_details	Order after ship/due	0	0	Pass (guard)
S11	crm_sales_details	Sales = Qty x Price	35	0	Pass
S12	erp_cust_az12	Birthdate range	31	**15**	**Not clean**
S13	erp_cust_az12	Gender values	messy	Male, Female, n/a	Pass
S14	erp_loc_a101	Country values	DE, US, USA, blanks	clean	Pass
S15	erp_px_cat_g1v2	Spaces	empty	empty	Pass
S16	erp_px_cat_g1v2	Maintenance	Yes, No	Yes, No	Pass

Row-count test (not in the file but worth adding): bronze versus silver counts. Verified: only `crm_cust_info` changes (18,494 to 18,484); the other five tables keep their row counts, which proves the cleansing repaired values instead of dropping rows.

What the silver tests cannot catch: whether a business decision was right (for example `ABS` on negative prices), values that are wrong but look valid (a wrong but plausible country), or referential problems between tables. Those need business confirmation and the cross-table tests in gold.

G3. Gold layer test cases

The gold test file has three queries. The first two test the **dimensions**, the third tests the **connections of the star schema**.

Test G1: `customer_key` is unique

```
SELECT customer_key, COUNT(*) AS duplicate_count
FROM gold.dim_customers
GROUP BY customer_key
HAVING COUNT(*) > 1;
```

- **How it works:** same bucket-and-count technique as S1 but on the **surrogate key**.
- **Why it matters:** the surrogate key is the dimension's primary key. If two rows share a key, any fact row pointing to it would join to **both**, doubling that sale. The usual cause is a join that fans out (one customer matching two ERP rows). The check is how you know the LEFT JOINs did not multiply rows.
- **Result: empty. Pass.** (The dimension has 18,484 rows, equal to the number of customers in silver.)

Test G2: `product_key` is unique

```
SELECT product_key, COUNT(*) AS duplicate_count
FROM gold.dim_products
GROUP BY product_key
HAVING COUNT(*) > 1;
```

Result: empty. Pass (295 rows, one per current product). Note: with `ROW_NUMBER()` generating the key, this test can never fail by construction, because row numbers are unique. Its real protective value is on the **business key**: add `GROUP BY product_number HAVING COUNT(*) > 1` to prove each product appears once, which is what stops the fact table from multiplying (the history problem that would give 89,818 fact rows).

Test G3: referential integrity, does every sale connect to a customer and a product?

```
SELECT *
FROM gold.fact_sales f
LEFT JOIN gold.dim_customers c ON c.customer_key = f.customer_key
LEFT JOIN gold.dim_products  p ON p.product_key  = f.product_key
WHERE p.product_key IS NULL OR c.customer_key IS NULL;
```

- **How it works:** start from every fact row, try to find its customer and product, and **return the rows where the lookup failed** (the dimension column is NULL).
- **Why LEFT JOIN is essential here:** an INNER JOIN would throw away the unmatched rows before the `WHERE` could see them, so the test would always pass, even for broken data. LEFT JOIN keeps them, so they can be found.
- **What a failure means:** a sale that cannot be attributed to a customer or product, so it would vanish from any report grouped by customer or product attributes.
- **Result: empty (0 orphan rows). Pass.**

Proof the test works (negative test). I deliberately removed 10 products from the dimension and re-ran the same logic: it returned **396 orphan sales rows**. So the test genuinely detects broken links; the empty result on the real data is meaningful.

G3 summary: gold test results

#	Test	Purpose	Result
G1	customer_key unique	Dimension primary key	Empty, pass
G2	product_key unique	Dimension primary key	Empty, pass
G3	Fact connects to both dimensions	Referential integrity	0 orphans, pass

What the gold tests do not cover, and what to add

The three tests pass, but they say nothing about these real findings:

Gap	Verified fact	Test to add
Products with no category	7 pedal products have NULL category	`SELECT * FROM gold.dim_products WHERE category IS NULL;`

Gap	Verified fact	Test to add
Sales with no order date	19 rows	`SELECT * FROM gold.fact_sales WHERE order_date IS NULL;`
Row count versus silver	60,398 in both	`SELECT (SELECT COUNT(*) FROM silver.crm_sales_details) - (SELECT COUNT(*) FROM gold.fact_sales);` expect 0
Totals match	not run	`SELECT (SELECT SUM(sls_sales) FROM silver.crm_sales_details) - (SELECT SUM(sales_amount) FROM gold.fact_sales);` expect 0
Business key unique in dimension	holds today	`SELECT product_number, COUNT(*) FROM gold.dim_products GROUP BY product_number HAVING COUNT(*) > 1;`
Allowed values	gender is Male, Female, n/a	`SELECT DISTINCT gender FROM gold.dim_customers;`

In short, a good test suite checks **keys, connections, counts, totals and allowed values**.

G4. How to explain testing in an interview

30-second answer: "I validated each layer with SQL checks that return rows only when data is wrong. In silver I checked duplicate and NULL keys, hidden spaces, standardised values, impossible dates, and the rule sales equals quantity times price. I ran them on bronze to find the problems and on silver to prove the fixes. In gold I checked that the surrogate keys are unique and that every fact row joins to a customer and a product, using LEFT JOINs so orphans are not hidden."

Likely cross-questions

1. **Why run the same tests on bronze and silver?** Bronze shows the problem exists (the test can fail); silver shows it is gone. A test that has never failed is unproven.
2. **Did all your tests pass?** All but one. The birthdate range test still returns 15 customers born before 1924, because the load only nulls future dates. I would either null them or relax the test, and document the decision.
3. **Which test is most important?** Sales = Quantity x Price, because it protects revenue, followed by the fact-to-dimension connectivity test.
4. **Why does the gold connectivity test use LEFT JOIN?** INNER JOIN would drop the orphans before the filter, so the test could never fail.
5. **Can a surrogate-key uniqueness test fail when the key comes from `ROW_NUMBER()`?** No, by construction. The useful uniqueness test is on the business key, because that is what prevents fan-out.

6. **What do your tests not cover?** Business correctness (for example whether negative prices are refunds), row-count and total reconciliation between layers, and the 7 products with no category.
7. **How would you automate them?** Put them in a test table or script, run them after each load, compare to expected counts, and make the pipeline fail (and alert) if any "expect no rows" check returns rows. Tools such as dbt tests or Great Expectations do this declaratively.
8. **How do you handle a failing test?** Decide whether it is a source problem (raise with the data owner), a transformation bug (fix the load) or an acceptable exception (document it and change the test).